IT'S ALL IN THE EMBEDDING! FAKE NEWS DETECTION USING DOCUMENT EMBEDDINGS

Ciprian-Octavian Truică[1], **Elena-Simona Apostol**[1]
[1] Computer Science and Engineering Department, Faculty of Automatic Control and Computers,
University Politehnica of Bucharest, Bucharest, Romania
ciprian.truica@upb.ro, elena.apostol@upb.ro

ABSTRACT

With the current shift in the mass media landscape from journalistic rigor to social media, personalized social media is becoming the new norm. Although the digitalization progress of the media brings many advantages, it also increases the risk of spreading disinformation, misinformation, and malformation through the use of fake news. The emergence of this harmful phenomenon has managed to polarize society and manipulate public opinion on particular topics, e.g., elections, vaccinations, etc. Such information propagated on social media can distort public perceptions and generate social unrest while lacking the rigor of traditional journalism. Natural Language Processing and Machine Learning techniques are essential for developing efficient tools that can detect fake news. Models that use the context of textual data are essential for resolving the fake news detection problem, as they manage to encode linguistic features within the vector representation of words. In this paper, we propose a new approach that uses document embeddings to build multiple models that accurately label news articles as reliable or fake. We also present a benchmark on different architectures that detect fake news using binary or multi-labeled classification. We evaluated the models on five large news corpora using accuracy, precision, and recall. We obtained better results than more complex state-of-the-art Deep Neural Network models. We observe that the most important factor for obtaining high accuracy is the document encoding, not the classification model's complexity.

Keywords fake news detection · document embeddings · deep learning · machine learning · text analysis · classification models

1 Introduction

With the increase in the digitalization of mass media, new journalistic paradigms for information distribution have emerged. These new paradigms have substantially changed the way society consumes information. By trying to be ahead of the competition, sometimes people who report on world events leave behind the rigors of classical journalism and publish their content as soon as possible in order to "go viral" by obtaining as many views, likes, comments, and shares as possible in a short amount of time [1]. This new paradigm centers on the users, catering to their needs, behavior, and interests. Along with the advantages the digitalization of mass media brings, it also increases the risk of misinformation, with potentially detrimental consequences for society, by facilitating the spread of misinformation [2, 3] in the form of fake news (which influenced the Brexit referendum [4], the 2016 US presidential election [5], COVID-19 vaccinations [6], etc.).

Fake news consists of news articles that are intentionally and verifiably false. This type of information aims to mislead readers by presenting alleged, real-seeming facts about social, economic, and political subjects of interest [7]. However, the current technological trends make this type of content harmful, with potentially dire consequences to the community (e.g., public polarization regarding elections). This has become a major challenge for democracy. Information propagated online may lack the rigor of classic journalism, and can, therefore, distort public perceptions, cause false alarms, and generate social unrest. Furthermore, the president of the EU, Ursula von der Leyen, has repeatedly condemned and asked for immediate action to be taken against the spread of fake news that undermines

democracy and public health [8]. Thus, the ideological polarization of readers through the spread of fake news is an important issue and requires scholarly attention. We believe that designing and building tools and methods for accurately detecting fake news is of great relevance, and thus, our results will have an overall positive impact.

In this paper, we propose a new approach that uses document embeddings (i.e., DOCEMB) for detecting fake news. We also present our benchmark on different architectures that detect fake news using binary or multi-labeled classification. The document embeddings are constructed using several (1) word embeddings trained on each dataset selected for experiments, and (2) pre-trained transformers. We employ TFIDF, word embeddings (i.e., WORD2VEC, FASTTEXT, and GLOVE), and transformers (i.e., BERT, ROBERTA, and BART) to create DOCEMB, our new document embedding approach. For classification, we train both classical Machine Learning models (i.e., Naïve Bayes, Gradient Boosted Trees) and Deep Learning models (i.e., Perceptron, Multi-Layer Perceptron, [Bi]LSTM, [Bi]GRU).

In our experiments, we analyze the performance of the DOCEMB based detection solution on multiple datasets annotated with either binary or multi-class labels. We use 4 binary datasets, i.e., a sample of 20 000 manually annotated news articles from the Fake News Corpus, Liar, Kaggle, and Buzz Feed News. Finally, we use 2 multi-labeled datasets, i.e., Liar with 6 labels and TSHP-17 with 3 labels. As evaluation metrics, we use accuracy, precision, and recall.

We compare our results with state-of-the-art Deep Neural Networks models. Our method outperforms these models on each dataset. The most important takeaway from our experiments is that we empirically show that:

(1) A simpler neural architecture offers better or at least similar results compared to complex architectures that employ multiple layers, and

(2) The difference in performance lies in the embeddings used to vectorize the textual data and how well these perform in encoding contextual and linguistic features.

The main contributions of this article are as follows:

(C_1) We propose a new document embedding (DOCEMB) constructed using word embeddings and transformers. We specifically trained the proposed DOCEMB on the five datasets used in the experiments.

(C_2) We show empirically that simple Machine Leaning algorithms trained with our proposed DOCEMB obtain similar results or better results than deep learning architectures specifically developed for the task of binary and multi-class fake news detection. This contribution is important in the machine learning literature because it changes the focus from the classification architecture to the document encoding architecture.

(C_3) We present a new manually filtered dataset. The original dataset is the widely used Fake News Corpus that was annotated with an automatic process.

This paper is structured as follows. Section 2 discusses current research on the topic of fake news detection. Section 3 introduces our approach and presents the different modules and models employed. Section 4 presents the datasets and analyzes the results. In Section 5, we summarize our key findings and discuss the major implications. Section 6 presents the conclusions and outlines directions for future work.

2 Related Work

As views and clicks monetize online media, for some publishers, it is most important to provide news that might interest their audience, to the detriment of the quality of the facts reported [9]. Thus, proper journalistic rigor has come under threat through the online spread of fake news.

Wang [10] employed SVM (Support Vector Machine), LogReg (Logistic Regression), BiLSTM (Bidirectional Long Short-Term Memory), and CNN (Convolutional Neural Network), to detect the veracity of $\sim$13 K short statements. The preprocessing was done using Google News' pre-trained WORD2VEC embeddings. Conroy et al. [11] present analysis methods based on linguistic and syntactic features for discovering fake news.

Many current approaches employ complex Deep Neural Network architectures, e.g., based on CNN (Convolutional Neural Network) [12, 13, 14], BiLSTM (Bidirectional Long Short-Term Memory) [15], and others. Ilie et al. [16] used multiple deep neural networks to determine how models that use pre-trained and specific trained word embeddings perform in the task of fake news detection. Further, some solutions use advanced document embeddings based on encoder architectures [17]. Kaliyar et al. [18] propose FakeBERT, an extension of FNDNet that uses BERT instead of GLOVE embeddings. Kula et al. [19] used a hybrid architecture for fake news detection that connects BERT with recurrent networks while Mersinias et al. [20] introduced CLDF, a new vectorization technique for extracting

feature vectors. The results for CLDF, FNDNet, and FakeBERT were obtained using the Kaggle dataset with ∼21 K news articles.

Different ensemble models have also been used for this task, with good results [21, 22]. Mondal et al. [21] used a voting-based ensemble method that relies on the voting of the collective majority. The authors employ only non-deep learning models and TF-IDF as the vectorization technique. Aslam et al. [22] used an ensemble-based deep learning model that combines two architectures, i.e., Bi-LSTM-GRU-Dense and Dense. Truică and Apostol [23] propose MisRoBÆRTa, a BERT- and RoBERTa-based ensemble model for fake news detection.

Sedik et al. [24] propose a deep learning approach that uses both sequential and recurrent layers. The sequential models employ stacked CNNs (i.e., CNN model) or concatenated CNN (i.e., C-CNN model), while the recurrent models use stacked CNN with LSTM and Dense layers (i.e., CNN-LSTM model) or simple GRU with a Dense layer (i.e., GRU model). The experimental results using the binary labeled Kaggle and Fake News Challenge dataset show that C-CNN and CNN-LSTM have the best performance, i.e., C-CNN obtains an accuracy of 99.90% on the Kaggle dataset, and CNN-LSTM obtains an accuracy of 96% on the Fake News Challenge dataset.

Several current solutions are based on linguistic and syntactic features, e.g., WELFake [25], which uses word embedding over linguistic features. In other current directions, multimodal learning that integrates comments [26], images [27], and the social and network context has been used [26, 28, 29]. Wang et al. [30] propose a knowledge-driven Multimodal Graph Convolutional Network model for detecting fake news from textual and visual information. This solution models posts from social media as graph data structures that combine textual and visual data with knowledge concepts.

Le and Mikolov [31] propose Doc2Vec as an extension to Word2Vec. Doc2Vec computes a feature vector for every document in the dataset, as opposed to Word2Vec, which computes every word in the dataset. Several articles have discussed the use of Doc2Vec for fake news detection, but it is used only as a baseline combined with traditional Machine Learning solutions. Cui et al. [32] use as a baseline Doc2Vec with SVM and compare it with graph-based Deep Learning solutions. Singh [33] presents several experiments on LIAR and Kaggle datasets using different vector space representations, i.e., one-hot encoding, TFIDF, Word2Vec, and Doc2Vec. Truică et al. [7] propose a BiLSTM architecture with Sentence Transformer for the fake news detection challenge at CheckThatLab! 2022. The proposed architecture uses BART for a monolingual fake news detection task and XML-RoBERTa for the multilingual task. For the multilingual task, the model relies on transfer learning. Thus, the BiLSTM XML-RoBERTa model is trained on English and tested on a German dataset. The proposed model managed to obtain an accuracy of 0.53 for the first task and an accuracy of 0.28 for the second task.

3 Methodology

Figure 1 presents the pipeline of our proposed solution. A labeled corpus of news articles is first preprocessed to extract the tokens. Then, the tokens are transformed into a vector model using term weighting schemes (TFIDF) and word/transformer embeddings. These vectors are used to create document embeddings. We also use the raw corpus to create document embeddings using transformers. The vectorized documents are then passed to the classification module. Finally, the classification is evaluated using accuracy, precision, and recall.

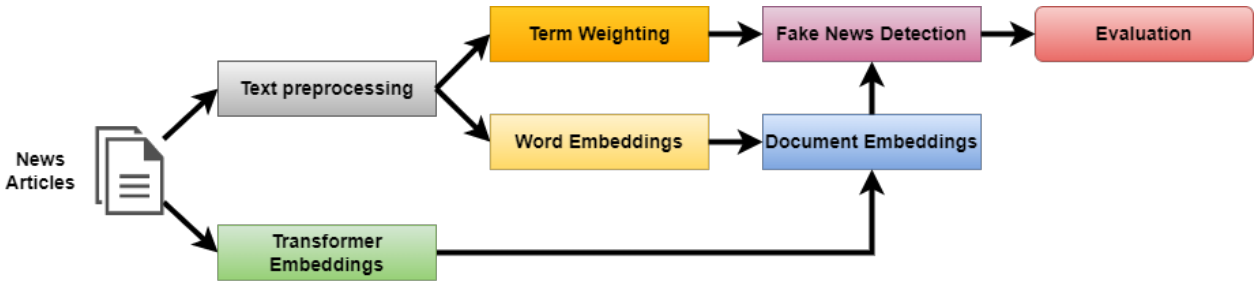

Figure 1: Proposed Pipeline.

3.1 Text Preprocessing

To prepare the text for vectorization, we use the following preprocessing steps to minimize the vocabulary and remove terms that bring no information gain [34]: (1) removal of punctuation and stopwords, and (2) word lemmatization. We

chose to lemmatize the words to minimize the vocabulary and remove any language inflections. We do not apply these preprocessing steps when using the transformer embeddings.

3.2 Term Weighting

To vectorize the preprocessed documents, we employ the TFIDF (Equation (1)). To compute this metric, we first need to compute the (1) term-frequency TF (Equation (2)) and (2) the inverse document frequency (Equation (3)). For a set of n documents $D = \{d_i \mid i \in \overline{1,n}\}$, we extract the set of m unique terms $V = \{t_j \mid j \in \overline{1,m}\}$. This set of unique terms is called a vocabulary. For each term, we compute the raw frequency (f_{t_j,d_i}), which counts the number of occurrences of a term t_j in a document d_i. The f_{t_j,d_i} does not store context and is biased towards longer documents. Thus, to remove the bias, we normalize the frequency with the length of the document ($\sum_{t' \in d_i} f_{t',d_i}$) and obtain TF [35]. Furthermore, to minimize the importance of common terms that bring no information value, the IDF (Equation (3)) is used to reduce the TF weight by a factor that grows with the collection frequency n_j of a term t_j, i.e., n_j is the number of documents where there is at least one occurrence of term t_j. Finally, to normalize TFIDF in the $[0, 1]$ range, we use the ℓ^2-norm (Equation (4)).

$$TFIDF(t_j, d_i, D) = \frac{TF(t_j, d_i) \cdot IDF(t_j, D)}{\ell^2(d_i)} \tag{1}$$

$$TF(t_j, d_i) = \frac{f_{t_j,d_i}}{\sum_{t' \in d_i} f_{t',d_i}} \tag{2}$$

$$IDF(t_j, D) = \log \frac{n}{n_j} \tag{3}$$

$$\ell^2(d_i) = \sqrt{\sum_{t \in d_i} \left(TF(t, d_i) \cdot IDF(t, D)\right)^2} \tag{4}$$

Using TFIDF, we construct a $n \times m$ document-term matrix $X = \{x_{ij} \mid i = \overline{1,n} \wedge j = \overline{1,m}\}$ ($X \in \mathbb{R}^{n \times m}$) where rows correspond to documents and columns to terms. The value $x_{ij} = TFIDF(t_j, d_i, D)$ represents the weight of term t_j in document d_i. Thus, each document d_i is represented by a vector $\mathbf{x}_i = \{x_{ij} \mid j \in \overline{1,m}\}$. For ease of notation, we use $\mathbf{x}_i$ for denoting lines in X.

3.3 Word Embeddings

Each word from the vocabulary is transformed into its vector representation. This module employs WORD2VEC [36, 37], FASTTEXT [38], and GLOVE [39]. For WORD2VEC and FASTTEXT, we use both the CBOW (Continuous Bag-of-Words) and SG (Skip-gram) models. By using these models, we obtain the embedding $WordEmb(t)$ for each term $t \in V$.

3.3.1 WORD2VEC

The WORD2VEC [36, 37] embedding model is used to create vectorized representations of the words in a dataset within the same vector space. This representation measures the distance between the corresponding vectors in this space to determine the context similarity. For WORD2VEC, there are two models for representing the words in this vector space: Continuous Bag-Of-Words (CBOW) or Skip-Gram.

CBOW Model The CBOW model attempts to predict a word using the context given by its surrounding words. Each word $t_i \in V$ ($i \in \overline{1,m}$) is defined by two d-dimensional (with $d \geq 2$ a natural number, i.e., $d \in \mathbb{N}$) vectors depending of its function in training: (1) $v_{t_i} \in \mathbb{R}^d$ is defined when t_i is used as the center word, and (2) $u_{t_i} \in \mathbb{R}^d$ is defined when t_i is used as a context word. The conditional probability of generating any center word t_c given its surrounding context words $\mathcal{T}_o = \{t_1, \ldots, t_{c-1}, t_{c+1}, \ldots, t_s\}$ within a context window of size s can be modeled by a probability distribution $p(t_c \mid \mathcal{T}_o)$ (Equation (5)) that considers the average of the context vectors $\overline{v}_o = \frac{1}{s}(v_{t_1} + \ldots + v_{t_{c-1}} + v_{t_{c+1}} + \ldots + v_{t_s})$.

$$p(t_c \mid \mathcal{T}_c) = \frac{e^{u_c^\top \overline{v}_o}}{\sum_{i=1}^{m} e^{u_i^\top \overline{v}_o}} \tag{5}$$

Skip-Gram Model The Skip-Gram model starts with the context word t_c as input and tries to generate its context. As in the CBOW case, the two d-dimensional vectors ($d \in \mathbb{N}$ and $d \geq 2$), i.e., $v_{t_i} \in \mathbb{R}^d$ and $u_{t_i} \in \mathbb{R}^d$, are defined for each word $t_i \in V$ ($i \in \overline{1,m}$). The conditional probability of generating any context word t_o given the center word t_c can be modeled by a softmax operation (Equation (6)).

$$p(t_o \mid t_c) = \frac{e^{u_o^\top v_c}}{\sum_{i=1}^{m} e^{u_i^\top v_c}} \tag{6}$$

3.3.2 FASTTEXT

FASTTEXT [38] is an extension to WORD2VEC and follows a similar approach to construct word embeddings [40]. The main difference between FASTTEXT and WORD2VEC is that FASTTEXT does not consider the word as the basic unit, but rather considers a bag of character n-grams. Using such an approach, the accuracy is improved, and the training time is decreased when compared to WORD2VEC. As in the case of WORD2VEC, FASTTEXT employs both CBOW and Skip-Gram models.

3.3.3 GLOVE

GLOVE (Global Vectors) [39] is another model used for creating word embeddings. To create the vector representation of words, GLOVE uses the word co-occurrences matrix. This matrix manages to encapsulate local and global corpus statistics regarding word–word co-occurrences. Thus, GLOVE for each word stores the frequency of its appearance in the same context as another word by employing a term co-occurrence matrix. Using the ratio of co-occurrence probability, GLOVE captures the relationship between words. Furthermore, GLOVE identifies word analogies and synonyms within the same contexts using this probability ratio.

3.4 Transformers Embeddings

To create transformer embeddings, we use BERT [41], ROBERTA [42], and BART [43]. By using these models, we obtain the word embedding by transformer $WordEmb(t)$ for each term $t \in V$.

3.4.1 BERT

BERT (bidirectional encoder representations from transformers) [41] is a deep bidirectional transformer architecture used for natural language understanding. Thus, in contrast to classic language models that treat textual data as unidirectional or bidirectional sequences of words, BERT learns contextual relations between the words by employing this deep bidirectional transformer architecture. Using the surrounding words of a given word, the model learns and creates a vector representation for each word that also encapsulates its context. Thus, BERT reads the entire sequence of words at once using the transformer encoder to create contextual word embeddings. By employing transfer learning, BERT can directly be used for various natural language processing operations, understanding, and generation. Furthermore, it can be fine-tuned by using new datasets to adapt to specific tasks. Experimental results on various tasks [41] show that the language models built with BERT manage to improve language context detection more than the models that use static word embeddings, which only see textual data as sequences of words.

3.4.2 ROBERTA

ROBERTA (a Robustly optimized BERT pre-training Approach) [42] is a training optimizing method for BERT. This model improves the language masking strategy of BERT by modifying the following key training aspects: (1) more data are used for training, (2) dynamic masking patterns instead of static masking patterns are employed, (3) the next-sentence pre-training objective is removed and replaced with full sentences without NSP (Next Sentence Prediction), (4) training is performed on longer sequences, (5) the mini-batches are improved, and (6) the learning rates are improved. Thus, all these modifications lead to improving ROBERTA's downstream task performance and mitigate some of the shortcomings encountered by the significantly undertrained BERT model.

3.4.3 BART

BART (bidirectional and autoregressive transformer) [43] is a transformer model that employs the standard transformer-based neural machine translation architecture, i.e., a generalized BERT architecture. The pre-training process of BART uses an arbitrary noising function to corrupt the textual data within the dataset in order to make the transformer learn how to recreate the original text during training. During the pre-training of BART, two key techniques are used to improve the words' contextual representations. Firstly, the order of original sentences is

randomly shuffled. Secondly, using a novel in-filling scheme, a single mask token is used to replace the spans of text. Experimental results [43] show that a fine-tuned BART works better than BERT for both text generation and comprehension tasks.

3.5 Document Embeddings

We create a vector for each document by averaging all the word or transformer embeddings for the words appearing in the document. Thus, if we have m_i terms in a document d_i, we obtain the document embedding (DOCEMB) x_i by summing all the embeddings $w(t)$ of the terms t that are present in document d_i as well as in the vocabulary V, and dividing the sum by m_i (Equation (7)). Each document embedding creates a context for words in a document and becomes an extension of the presented word embeddings.

$$\mathbf{x}_i = \frac{\sum_{t \in d_i} w(t)}{m_i} \tag{7}$$

Similarly to TFIDF, we construct a document-embedding matrix $X = \{\mathbf{x}_i \mid i = \overline{1,n}\}$ where each row corresponds to the document embedding $\mathbf{x}_i$. For this matrix, the columns are not associated to terms in the vocabulary V, and the number of columns is different from the total number of terms in V. In this case, m is the size of the embedding vector. For ease of notation, we use m as the number of columns, although it is different than the number of terms in the vocabulary, as in the case of the document-term matrix. Thus, $X \in \mathbb{R}^{n \times m}$ is a $n \times m$ matrix.

3.6 Fake News Detection

Classification is used to determine the veracity of a news article, i.e, fake news detection. Given a set of documents D represented by the matrix $X \in \mathbb{R}^{n \times m}$ (either the document-terms or the document embedding matrix), a set of classes $Y = \{y_1, \ldots, y_n\}$ with values in a discrete domain $C = \{c_k \mid k = \overline{1,\kappa}\}$ ($Y \subseteq C$) of size κ (i.e., the number of classes is κ), and an implication $\mathbf{x}_i \rightarrow y_i$ ($i \in \overline{1,n}$), the objective of classification is to predict $\hat{y}_i = f(\mathbf{x}_i)$ ($\hat{y}_i \in \hat{Y} \subseteq C$) that best approximates y_i. For the fake news detection task, we employ the following algorithms to construct models: Naïve Bayes (NB), Gradient Boosted Trees (XGBTrees), Perceptron, Multi-Layer Perceptron (MLP), Long Short-Term Memory Network (LSTM), Bidirectional LSTM (BiLSTM), Gated Recurrent Units (GRU), and Bidirectional GRU (BiGRU). For comparison, we use MisRoBÆRTa [23]. In the original article that presents MisRoBÆRTa, the authors fine-tune both BART and ROBERTA. In this work, we use the pre-trained BART (*facebook/bart-large*) and ROBERTA (*roberta-base*) from HuggingFace [44].

3.6.1 Naïve Bayes

The Naïve Bayes (NB) model is a probabilistic classification algorithm that computes the probability of $\mathbf{x}_i$ given a class c_k (Equation (8)), where $p(\mathbf{x}_i)$ and $p(c_k)$ are the probability of a document and a class, respectively, and $p(\mathbf{x}_i \mid c_k)$ is the probability of class c_k given $\mathbf{x}_i$. Expending $\mathbf{x}_i$ by its components $\{x_{i1}, \ldots, x_{im}\}$, we can rewrite Equation (8) as Equation (9).

$$p(c_k \mid \mathbf{x}_i) = \frac{p(c_k)p(\mathbf{x}_i \mid c_k)}{p(\mathbf{x}_i)} \tag{8}$$

$$p(c_k \mid x_{i1}, \ldots, x_{im}) = \frac{p(c_k)p(x_{i1}, \ldots, x_{im} \mid c_k)}{p(\mathbf{x}_i)} \tag{9}$$

The denominator $p(\mathbf{x}_i)$ is constant, while $p(x_{i1}, \ldots, x_{im} \mid c_k)$ is equivalent to the joint probability $p(c_k, x_{i1}, \ldots, x_{im})$. Furthermore, all the terms are conditionally independent given a class c_k. Thus, $p(c_k, x_{i1}, \ldots, x_{im}) = \prod_{j=1}^{m} p(x_{ij} \mid c_k)$. Using these assumptions, the Naïve Bayes classifier tries to estimate the class $\hat{y}_i$ using Equation (10).

$$\hat{y}_i = \underset{c_k \in C}{\operatorname{argmax}}\, p(c_k) \prod_{j=1}^{m} p(x_{ij} \mid c_k) \tag{10}$$

There are various types of Naïve Bayes classifiers; the most common ones are Multinomial Naïve Bayes and Gaussian Naïve Bayes.

Multinomial Naïve Bayes Multinomial Naïve Bayes (MNB) models the distribution of words in a document by using a multinomial representation for the distribution of probabilities that a word appears for a certain class (Equation (11)). The assumption for this model is that a document is handled as a sequence of words. Also, it is assumed that each word position is generated independently of every other [45].

$$p(\mathbf{x}_i \mid c_k) = \frac{(\sum_{j=1}^{m} x_{ij})!}{\prod_{j=1}^{m} x_{ij}!} \prod_{j=1}^{m} p(x_{ij} \mid c_k)^{x_{ij}} \tag{11}$$

Equation (12) presents the Multinomial Naïve Bayes classification model.

$$\hat{y}_i = \underset{c_k \in C}{\operatorname{argmax}}\, p(c_k) \frac{(\sum_{j=1}^{m} x_{ij})!}{\prod_{j=1}^{m} x_{ij}!} \prod_{j=1}^{m} p(x_{ij} \mid c_k)^{x_{ij}} \tag{12}$$

Gaussian Naïve Bayes The Gaussian Naïve Bayes (GNB) model is used when dealing with continuous data. The model is based on the assumption that continuous values correlated with each class are distributed according to a Gaussian distribution. Thus, given column $j \in \overline{1,m}$ from X and a class c_k, we employ the following steps:

- Segment the data by class c_k.
- Compute the associated means μ_j and variances σ_j of dimension j using the values x_{ij} ($i \in \overline{1,n}$), only for the lines $\mathbf{x}_i \in X$ labeled with class c_k.
- Compute the probability $p(x_{ij} \mid c_k)$ (Equation (13)).

$$p(x_{ij} \mid c_k) = \frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-\frac{x_{ij}-\mu_j^2}{2\sigma_j^2}} \tag{13}$$

Equation (14) presents the Gaussian Naïve Bayes classifier.

$$\hat{y}_i = \underset{c_k \in C}{\operatorname{argmax}}\, p(c_k) \prod_{j=1}^{m} \frac{1}{\sqrt{2\pi\sigma_j^2}} e^{-\frac{x_{ij}-\mu_j^2}{2\sigma_j^2}} \tag{14}$$

3.6.2 Gradient Boosted Trees

Gradient boosting is an ensemble method that uses multiple weak predictions learners. In the case of Gradient Boosted Trees, the weak learners are Decision Trees. Similar to other classification methods, the method tries to predict $\hat{y}_i = f(\mathbf{x}_i) = \mathbf{w} \cdot \mathbf{x}_i + b$ that best approximates the true class y_i of $\mathbf{x}_i$ by minimizing an objective function $L(\hat{Y}_i, Y_i)$ that represents the training loss, e.g., the mean score $L(y_i, \hat{y}_i) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 = \frac{1}{n}\sum_{i=1}^{n} l(\hat{y}_i, y_i)$. As the model builds T weak learners $f^{(t)}(\mathbf{x}_i)$ ($t \in \overline{1,T}$), at each stage $t \in T$ the model tries to determine $\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f^{(t)}(\mathbf{x}_i)$ using the previously estimated value $\hat{y}_i^{(t-1)}$ and the function $f^{(t)}(\mathbf{x}_i)$ determined by the current weak learner that best fits the residuals, i.e., $f^{(t)}(\mathbf{x}_i) = y_i - \hat{y}_i^{(t)}$. As the objective is to minimize training loss and to obtain the specific objective at step t,we can take the Taylor expansion of the loss function up to the second order for each learner and remove all the constants to obtain $L^{(t)}(\hat{Y}_i^{(t)}, Y_i)$ (Equation (15)).

$$\begin{aligned} L^{(t)}(\hat{Y}_i^{(t)}, Y_i) &= \sum_{i=1}^{n} [g_i f^{(t)}(\mathbf{x}_i) + \frac{1}{2} h_i (f^{(t)}(\mathbf{x}_i))^2] \\ g_i &= \partial_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)}) \\ h_i &= \partial^2_{\hat{y}_i^{(t-1)}} l(y_i, \hat{y}_i^{(t-1)}) \end{aligned} \tag{15}$$

3.6.3 Perceptron

The Perceptron model (Equation 16) is a simple non-linear processing unit that tries to predict the label $\hat{y}_i$ for a given input x_i by adjusting a weight vector $\mathbf{w} \in \mathbb{R}^m$ using the sigmoid activation $\delta_s(z) = \frac{1}{1+e^{-z}} \in [0,1]$. The objective for a good prediction is to minimize the average cross-entropy loss function between the set of prediction $\hat{Y}$ and the set of true labels Y (Equation 17).

$$\hat{y}_i = \delta_s(\mathbf{w} \cdot \mathbf{x}_i + b) \tag{16}$$

$$L(\hat{Y}, Y) = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)) \tag{17}$$

3.6.4 Multi-Layer Perceptron

The Multi-Layer Perceptron (MLP) model is a Deep Learning architecture that stacks multiple layers $j \in \overline{1, l}$ of fully-connected Perceptron units. The MLP architecture can be divided into three components: (1) the input i layer ($j = 1$), (2) the hidden layers $j \in \overline{2, l-1}$, and (3) the output layer $o = \hat{y}$ ($j = l$). Each node in layer j connects to every node in the following layer $j + 1$ with a certain weight W_j. Because the connections between the layers are directed from the input i to the output o by passing information through the hidden layers h_j, the MLP model is a feed-forward architecture. Equation (18) presents the MLP classification model at a given iteration t.

$$\begin{aligned} i^{(t)} &= \delta_s(W_1 \cdot \mathbf{x}_i^{(t)} + b_1) \\ h_j^{(t)} &= \delta_s(W_j \cdot h_{j-1}^{(t)} + b_j) \\ o^{(t)} &= \delta_s(W_l \cdot h_{l-1}^{(t)} + b_l) \end{aligned} \tag{18}$$

3.6.5 Long Short-Term Memory

Long Short-Term Memory (LSTM) [46] is a Recurrent Artificial Neural Network that uses two state components for classification. The first component, represented by a hidden state, is the short-term memory that learns the short-term dependencies between the previous and current states. The second component, represented by an internal cell state, is the long-term memory which stores long-term dependencies between the previous and current states. The model uses three gates to preserve the long-term memory within the state: (1) input gate ($i \in \mathbb{R}^n$), (2) forget gate ($f \in \mathbb{R}^n$), and (3) output gate ($o \in \mathbb{R}^n$). Equation (19) presents the compact forms for the state updates of the LSTM unit for a given iteration t, where:

- $\mathbf{x}_i^{(t)} \in \mathbb{R}^m$ is the input vector of dimension m at step t, with $\mathbf{x}_i^{(0)} = \mathbf{x}_i \in X$;
- $h^{(t)} \in \mathbb{R}^n$ is the hidden state vector as well as the unit's output vector of dimension n, where the initial value is $h^{(0)} = 0$;
- $\tilde{c}^{(t)} \in \mathbb{R}^n$ is the input activation vector;
- $c^{(t)} \in \mathbb{R}^n$ is the cell state vector, with the initial value $c^{(0)} = 0$;
- $W_i, W_f, W_o, W_c \in \mathbb{R}^{n \times m}$ are the weight matrices corresponding to the current input of the input gate, output gate, forget gate, and the cell state;
- $V_i, V_f, V_o, V_c \in \mathbb{R}^{n \times n}$ are the weight matrices corresponding to the hidden output of the previous state for the current input of the input gate, output gate, forget gate, and the cell state;
- $b_i, b_f, b_o, b_c \in \mathbb{R}^n$ are the bias vectors corresponding to the current input of the input gate, output gate, forget gate, and the cell state;
- $\delta_h(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in [-1, 1]$ is the hyperbolic tangent activation function;
- $\odot$ is the Hadamard Product, i.e., element wise product.

$$\begin{aligned} i^{(t)} &= \delta_s(W_i \mathbf{x}_i^{(t)} + V_i h^{(t-1)} + b_i) \\ f^{(t)} &= \delta_s(W_f \mathbf{x}_i^{(t)} + V_f h^{(t-1)} + b_f) \\ o^{(t)} &= \delta_s(W_o \mathbf{x}_i^{(t)} + V_o h^{(t-1)} + b_o) \\ \tilde{c}^{(t)} &= \delta_h(W_c \mathbf{x}_i^{(t)} + V_c h^{(t-1)} + b_c) \\ c^{(t)} &= i^{(t)} \odot \tilde{c}^{(t)} + f^{(t)} \odot c^{(t-1)} \\ h^{(t)} &= o^{(t)} \odot \delta_h(c^{(t)}) \end{aligned} \tag{19}$$

We chose LSTM because it manages to avoid the vanishing and the exploding gradient issues by regulating the way the recurrent weights are learned.

3.6.6 Bidirectional LSTM

As the LSTM model processes sequence data, it is able to capture *past* information. To take into consideration *future* information as well, we use the Bidirectional LSTM (BiLSTM). The BiLSTM encapsulates *past* and *future* information through the use of two hidden states (Equation (20)), where (1) $\overrightarrow{h}^{(t)}$ processes the input in a forward manner using the past information provided by the forward LSTM ($\overrightarrow{LSTM}_F$), and (2) $\overleftarrow{h}^{(t)}$ processes the input in a backward manner using the future information provided by the backward LSTM ($\overleftarrow{LSTM}_B$).

$$\begin{aligned} \overrightarrow{h}^{(t)} &= \overrightarrow{LSTM}_F(\mathbf{x}_i^{(t)}) \\ \overleftarrow{h}^{(t)} &= \overleftarrow{LSTM}_B(\mathbf{x}_i^{(t)}) \end{aligned} \tag{20}$$

At every time-step, the hidden states, i.e., $\overrightarrow{h}^{(t)}$ and $\overleftarrow{h}^{(t)}$, are concatenated into one hidden state $h'^{(t)}$ (Equation (21)). This approach enables the encoding of information from both past and future contexts in the hidden state $h'^{(t)}$.

$$h'^{(t)} = [\overrightarrow{h}^{(t)} \,||\, \overleftarrow{h}^{(t)}] \tag{21}$$

3.6.7 Gated Recurrent Unit

The Gated Recurrent Unit (GRU) [47] is a Recurrent Artificial Neural Network that simplifies the LSTM unit and improves performance considerably. Instead of three gates as in the case of LSTM, the GRU has only two gating mechanisms. The first gating mechanism is the update gate ($u \in \mathbb{R}^n$). This gate encodes both the forget gate and the input gate that are present in the LSTM cell. The second gating mechanism is the reset gate ($r \in \mathbb{R}^n$). This gate determines the percentage of information from the previous hidden state that contributes to the candidate state of the new step [48] Furthermore, the GRU uses the hidden state as the only state component. Equation (22) presents the compact forms for the state updates of the GRU unit at a given iteration step t, where:

- $\mathbf{x}_i^{(t)} \in \mathbb{R}^m$ is the input vector of dimension m at step t, with $\mathbf{x}_i^{(0)} = \mathbf{x}_i \in X$;
- $i^{(t)} \in \mathbb{R}^n$ is the input and output of the cell at step t;
- $\tilde{h}^{(t)} \in \mathbb{R}^n$ is the candidate hidden state with a cell dimension of n;
- $h^{(t)} \in \mathbb{R}^n$ is the current hidden state with a cell dimension of n;
- $W_u, W_r, W_h \in \mathbb{R}^{n \times m}$ are the weight matrices corresponding to the current input of the update gate, reset gate, and the hidden state;
- $V_u, V_r, V_h \in \mathbb{R}^{n \times m}$ are the weight matrices corresponding to the hidden output of the previous state for the current input of the update gate, reset gate, and the hidden state;
- $b_u, b_r, b_h \in \mathbb{R}^n$ are the bias vectors corresponding to the current input of the update gate, reset gate, and the hidden state;
- $\odot$ is the Hadamard Product.

$$\begin{aligned} u^{(t)} &= \delta_s(W_u \mathbf{x}_i^{(t)} + V_u h^{(t-1)} + b_u) \\ r^{(t)} &= \delta_s(W_r \mathbf{x}_i^{(t)} + V_r h^{(t-1)} + b_r) \\ \tilde{h}^{(t)} &= \delta_h(W_h \mathbf{x}_i^{(t)} + V_h h^{(t-1)} + b_h) \\ h^{(t)} &= u^{(t)} \odot \tilde{h}^{(t)} + (1 - u^{(t)}) \odot h^{(t-1)} \end{aligned} \tag{22}$$

3.6.8 Bidirectional GRU

Similar to the BiLSTM, the Bidirectional GRU (BiGRU) considers both *past* and *future* information by employing a forward and backward GRU, i.e., $\overrightarrow{GRU}_F$ and $\overleftarrow{GRU}_B$, respectively. The $\overrightarrow{GRU}_F$ and $\overleftarrow{GRU}_B$ are associated to two hidden states (Equation (23)): (1) $\overrightarrow{h}^{(t)}$ which processes the input in a forward manner using $\overrightarrow{GRU}_F$, and (2) $\overleftarrow{h}^{(t)}$ which processes the input in a backwards manner using $\overleftarrow{GRU}_B$. As for BiLSTM, the hidden states $\overrightarrow{h}^{(t)}$ and $\overleftarrow{h}^{(t)}$ are concatenated at every time-step to encode the information from both past and future contexts into one hidden state $h'^{(t)} = [\overrightarrow{h}^{(t)} \,||\, \overleftarrow{h}^{(t)}]$.

$$\begin{aligned} \overrightarrow{h}^{(t)} &= \overrightarrow{GRU}_F(\mathbf{x}_i) \\ \overleftarrow{h}^{(t)} &= \overleftarrow{GRU}_B(\mathbf{x}_i) \end{aligned} \tag{23}$$

3.7 Evaluation Module

We use accuracy, precision, and recall [49] to evaluate the models. For binary classification with the classes positive and negative, the following information is used to construct a confusion matrix that is afterward used to compute the evaluation metrics:

- tp (True Positive) is the number of positive observations that are correctly classified;
- fn (False Negative) is the number of positive observations that are incorrectly classified as negative;
- fp (False Positive) is the number of false observations that are incorrectly classified as positive;
- tn (True Negative) is the number of false observations that are correctly classified.

Accuracy (Equation (24)) measures the overall effectiveness of a classifier. Precision (Equation (25)) measures the class agreement of the data labels within the positive labels. Recall (Equation (26)) measures the effectiveness of a classifier in identifying positive labels.

$$A = \frac{tp + tn}{tp + tn + fp + fn} \tag{24}$$

$$P = \frac{tp}{tp + fp} \tag{25}$$

$$R = \frac{tp}{tp + fn} \tag{26}$$

4 Experimental Results

In this section, we present the experimental results obtained using our methodology. Firstly, we introduce a human-verified sample from the Fake News Corpus [50] and present the results of the exploratory data analysis performed on it. Secondly, we present the experimental setup for our experiments as well as the hyperparameters and implementation packages for the models. Thirdly, we present the experimental results using the different sentence embeddings and classification methods on the Fake News Corpus sample. Lastly, we show the generalization of our observation by performing additional experiments on 5 additional datasets: LIAR multiclass [10], LIAR binary [51], Kaggle [12, 18], Buzz Feed News [52], and TSHP-17 [53, 54].

4.1 Dataset Details

For the experiments, we used a set of 20K English language news articles (10K reliable and 10K fake) selected from the Fake News Corpus [50] as it is widely used in current research [55, 56, 57, 16, 23]. Some of the labels might not be correct because the original dataset is not manually annotated. However, this shortcoming should not pose a practical issue for classification models, as ML/DL models generalize better when noise is added [58]. Instead, this should help the models to better generalize and remove overfitting. Additionally, we made sure that URL for the selected article point to the correct article by matching the titles and authors.

Before performing the experiments, we verified the label correctness for the sampled news articles using computer science students as annotators. In total, there were 40 student annotators to annotate 25K articles (12.5K reliable and 12.5K fake). We sampled more articles to mitigate any inconsistencies between two annotators as well as between the final annotation and the original label. For their annotation work, the students obtained credits for different courses.

Before annotating the articles, the students received an instruction list that explains the annotation task. The annotation task included the following steps:

(1) Verify that the title matches the title from the URL;

(2) Verify that the content matches the content from the URL;

(3) Verify that the authors match the authors from the URL;

(4) Verify that the source matches the source from the URL;

(5) Verify if the information is false or reliable;

Each article was manually verified by two annotators. If there was no consensus between the two, a third annotator was used to break the tie. In 99% of the cases, there was no requirement for adding a third annotator. In the experiments,

we removed all the articles where no consensus was found as well as where a difference between the human annotation and the original label was found. In the end, we scaled down the sample to $20K$ news articles.

Table 1 presents the corpus statistics and information before and after preprocessing. We observe that, although there is a small imbalance in the number of tokens between the classes, this imbalance is small enough not to add bias to the classification task. We also extracted the top 10 unigrams and the top topic using the NMF algorithm for topic modeling [59]. We used the *NLTK* [60] for extracting unigrams and *scikit-learn* [61] for NMF. We computed the average similarity with *PolyFuzz* [62] by employing the pre-trained FASTTEXT embedding on news articles (**sim(FT)**) and the base-case BERT (**sim(BERT)**). Analyzing both similarities, we conclude that the documents discuss the same topics (Table 1). For the neural networks (i.e., Perceptron, MLP, LSTM, BiLSTM, GRU, BiGRU), we use one-hot encoders to represent the labels.

Table 1: Dataset description and statistics and information

Statistics before preprocessing			**#Tokens per document**				**#Tokens per Class**	
Class	**Encoding**	**Description**	**Mean**	**Min**	**Max**	**StdDev**	**Unique**	**All**
Fake News	1	Fabricated or distorted information	517.83	6	8 812	883.35	119 283	5 178 300
Reliable	0	Reliable information	575.66	7	10 541	602.16	82 203	5 756 643
Entire dataset statistics			546.75	6	10 541	618.63	159 113	10 934 943

Textual information after preprocessing			**sim(FT)**	**sim(BERT)**
Top-10 Unigrams	**Fake News**	people time government world year story market American God day	**0.83**	**0.93**
	Reliable	people God Christian government American time world war America political		
Top-1 Topic	**Fake News**	people Trump year day government time state world market war	**0.84**	**0.94**
	Reliable	church Trump people God president war state year Bush government		

4.2 Experimental Setup

In our experiments, we analyzed how well we can predict if an article is fake or reliable using multiple vectorizations: (1) the TFIDF vector space model, and (2) eight document embeddings (DOCEMB) constructed using five word embeddings (WORD2VEC CBOW, WORD2VEC SG, FASTTEXT CBOW, FASTTEXT SG, and GLOVE), and three transformers embeddings (BERT, ROBERTA, and BART). We trained our own word embeddings and TFIDF vectorizer. We trained each word embedding for 100 epochs using a window size of 10, a learning rate of 0.05, and a size of 128. For TFIDF, we ignored words that appeared in less than 4 documents and kept the top 5K relevant features. We used *SpaCy* for preprocessing, *scikit-learn* for implementing TFIDF, *gensim* [63] for the WORD2VEC and FASTTEXT models, and the *python-glove* [64] package for GLOVE. For the transformer embeddings, we used the pre-trained uncased large versions from *HuggingFace* [44] together with *SimpleTransformers* [65] and *SentenceTransformers* [66].

For the experiments, we used the following algorithms for classification: Naïve Bayes (NB), Perceptron, Multi-layer Perceptron (MLP), LSTM, Bidirectional LSTM (BiLSTM), GRU, and Bidirectional GRU (BiGRU). We used *scikit-learn* for implementing NB. We applied the Multinomial NB for the TFIDF experiments as documents can get sparse, and the Gaussian NB for the transformer and word embeddings experiments as the embeddings can have negative values. For the Gradient Boosted Trees, we used the XGBoost Python library [67].

The neural-based fake news detection module used Multi-layer Perceptron, LSTM, Bidirectional LSTM, GRU, and Bidirectional GRU. Each layer consists of 100 cells. The LSTM was configured as in [46], while the GRU was configured as presented in [47]. A Dense layer with 2 units and a sigmoid function as activation was used as the output.

For the LSTM, BiLSTM, GRU, and BiGRU models, we used the ADAM optimizer and a 64-batch size. All the neural network models were trained for 100 epochs with an Early-Stopping mechanism to mitigate overfitting. We employed *Keras* for implementing the neural models. For comparison, we used the free implementation of MisRoBÆRTa [23], made available by the authors on GitHub.

The code will be made available on GitHub upon the acceptance of this work.

4.3 Fake News Detection

For the experiments, we used an NVIDIA® DGX Station™. Table 2 presents the results for the fake news detection task. We tested the models in 10 rounds and used a 70%–10%–20% train–validation–test split ratio. Each split shuffles the dataset and extracts a stratified sample initialized with different random seeds. We report the average and standard deviation for each metric.

Table 2: Fake news detection results on the sample extracted from the Fake News Corpus

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	92.69 ± 0.25	91.72 ± 0.39	93.86 ± 0.41	98.76 ± 0.21	99.79 ± 0.08	97.72 ± 0.43
DocEmb Word2Vec CBOW	66.29 ± 0.53	60.78 ± 0.42	91.85 ± 0.27	95.67 ± 0.18	96.17 ± 0.37	95.13 ± 0.34
DocEmb Word2Vec SG	53.10 ± 0.37	51.74 ± 0.20	92.46 ± 0.77	97.10 ± 0.21	97.61 ± 0.31	96.56 ± 0.16
DocEmb FastText CBOW	56.13 ± 0.53	53.59 ± 0.33	91.45 ± 0.58	94.90 ± 0.24	95.49 ± 0.39	94.24 ± 0.46
DocEmb FastText SG	54.00 ± 0.69	52.23 ± 0.38	93.66 ± 0.98	97.05 ± 0.26	97.45 ± 0.32	96.64 ± 0.49
DocEmb GloVe	53.43 ± 0.42	51.98 ± 0.24	89.94 ± 0.78	96.02 ± 0.30	96.73 ± 0.35	95.26 ± 0.43
DocEmb BERT	80.90 ± 0.64	74.94 ± 0.67	92.87 ± 0.58	97.43 ± 0.21	97.75 ± 0.29	97.10 ± 0.30
DocEmb RoBERTa	91.98 ± 0.31	94.05 ± 0.44	89.63 ± 0.60	97.38 ± 0.22	98.72 ± 0.31	96.02 ± 0.42
DocEmb BART	89.13 ± 0.32	83.71 ± 0.46	97.19 ± 0.37	98.26 ± 0.19	98.18 ± 0.31	98.35 ± 0.21
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	95.79 ± 0.33	96.55 ± 0.46	94.98 ± 0.42	98.04 ± 0.19	98.37 ± 0.28	97.70 ± 0.33
DocEmb Word2Vec CBOW	93.61 ± 0.27	94.22 ± 0.53	92.93 ± 0.49	94.96 ± 0.31	95.26 ± 0.36	94.63 ± 0.64
DocEmb Word2Vec SG	92.04 ± 0.34	94.65 ± 0.49	89.12 ± 0.91	95.88 ± 0.30	96.34 ± 0.65	95.40 ± 1.09
DocEmb FastText CBOW	93.46 ± 0.30	94.48 ± 0.64	92.33 ± 0.67	94.92 ± 0.23	95.12 ± 0.72	94.71 ± 0.67
DocEmb FastText SG	91.60 ± 0.49	94.06 ± 0.54	88.81 ± 0.76	96.00 ± 0.30	96.48 ± 0.35	95.48 ± 0.74
DocEmb GloVe	89.57 ± 0.50	92.57 ± 0.60	86.04 ± 1.18	94.05 ± 0.38	94.29 ± 0.89	93.79 ± 0.60
DocEmb BERT	97.09 ± 0.21	97.50 ± 0.62	96.66 ± 0.62	98.34 ± 0.19	98.56 ± 0.62	98.11 ± 0.56
DocEmb RoBERTa	96.19 ± 0.50	96.89 ± 1.62	95.49 ± 0.99	97.28 ± 0.55	98.51 ± 1.32	96.04 ± 0.47
DocEmb BART	98.57 ± 0.15	98.71 ± 0.29	98.43 ± 0.33	98.93 ± 0.16	99.07 ± 0.55	98.80 ± 0.39
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	97.88 ± 0.23	98.20 ± 0.19	97.55 ± 0.40	97.84 ± 0.24	98.21 ± 0.30	97.46 ± 0.49
DocEmb Word2Vec CBOW	96.59 ± 0.36	96.38 ± 1.05	96.84 ± 1.17	96.89 ± 0.26	96.89 ± 0.53	96.89 ± 0.38
DocEmb Word2Vec SG	96.23 ± 0.31	96.70 ± 0.97	95.76 ± 1.32	96.39 ± 0.37	96.81 ± 1.30	95.98 ± 1.32
DocEmb FastText CBOW	96.16 ± 0.26	96.22 ± 0.55	96.11 ± 0.85	96.30 ± 0.32	96.63 ± 1.19	95.98 ± 1.14
DocEmb FastText SG	96.61 ± 0.27	96.52 ± 0.91	96.72 ± 0.90	96.79 ± 0.22	96.78 ± 0.91	96.82 ± 0.88
DocEmb GloVe	94.66 ± 0.48	94.92 ± 2.02	94.45 ± 1.72	94.86 ± 0.37	95.24 ± 1.67	94.51 ± 1.63
DocEmb BERT	98.57 ± 0.34	98.59 ± 0.76	98.57 ± 1.10	98.72 ± 0.40	98.90 ± 0.81	98.55 ± 0.87
DocEmb RoBERTa	96.88 ± 1.57	98.02 ± 2.95	95.80 ± 1.56	96.97 ± 1.33	97.78 ± 2.85	96.22 ± 0.74
DocEmb BART	99.29 ± 0.10	99.46 ± 0.13	99.13 ± 0.14	99.34 ± 0.08	99.48 ± 0.12	99.20 ± 0.11
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	97.88 ± 0.24	98.28 ± 0.27	97.47 ± 0.45	97.84 ± 0.30	98.04 ± 0.60	97.64 ± 0.58
DocEmb Word2Vec CBOW	96.56 ± 0.29	96.43 ± 0.75	96.71 ± 1.01	96.57 ± 0.23	96.37 ± 0.90	96.81 ± 0.69
DocEmb Word2Vec SG	96.21 ± 0.44	95.91 ± 1.44	96.58 ± 0.97	96.35 ± 0.38	96.45 ± 1.09	96.26 ± 1.47
DocEmb FastText CBOW	96.12 ± 0.16	96.54 ± 1.04	95.70 ± 1.00	96.20 ± 0.33	96.47 ± 0.88	95.91 ± 1.00
DocEmb FastText SG	96.40 ± 0.35	96.11 ± 1.22	96.74 ± 1.32	96.76 ± 0.22	96.76 ± 0.59	96.77 ± 0.50
DocEmb GloVe	94.62 ± 0.64	95.60 ± 2.30	93.66 ± 1.81	94.84 ± 0.60	95.80 ± 1.74	93.86 ± 2.01
DocEmb BERT	98.82 ± 0.11	98.71 ± 0.52	98.92 ± 0.48	98.61 ± 0.41	99.17 ± 0.47	98.05 ± 1.14
DocEmb RoBERTa	97.37 ± 0.60	99.17 ± 0.43	95.56 ± 1.47	97.31 ± 0.44	98.34 ± 1.24	96.26 ± 0.72
DocEmb BART	99.31 ± 0.10	99.39 ± 0.23	99.22 ± 0.14	**99.36 ± 0.08**	99.50 ± 0.09	99.22 ± 0.09
	Accuracy		**Precision**		**Recall**	
MisRoBÆRTa [23]	99.34 ± 0.03		99.34 ± 0.03		99.34 ± 0.02	

The overall best performance (i.e., an accuracy of 99.36%) was obtained with the BiGRU model when employing the document embeddings obtained with BART. The BiLSTM model obtained similar accuracy as when using BART, i.e., 99.36%. We observe that, regardless of the model, the best results are obtained when employing the document embeddings created with BART.

Regardless of the model, TFIDF obtained very good results. For some models, it even outperformed the document embeddings obtained from the word or transformer embeddings, e.g., Naïve Bayes with TFIDF obtained an accuracy of 92.69% while Naïve Bayes with RoBERTa obtained an accuracy of 91.98%, Gradient Boosted Trees with TFIDF obtained an accuracy of 98.76% while Gradient Boosted Trees with RoBERTa obtained an accuracy of 98.72%. It is worth noting that the worst results were obtained with Naïve Bayes when employing the document embeddings created using the word embeddings.

Taking a closer look at the neural networks, we observe that the model ranking when considering accuracy is as follows: (1) BiGRU, (2) BiLSTM, (3) GRU, (4) LSTM, (5) Multi-Layer Perceptron, and (6) Perceptron. We note that LSTM and GRU obtained similar results with regard to document embedding, i.e., the results obtained by LSTM with

the document embedding employing WORD2VEC CBOW are very similar to the results obtained by GRU with the same document embedding model. The same observation holds when comparing BiLSTM and BiGRU. Moreover, the neural models that use document embeddings created using word embeddings were outperformed by the ones using transformer embeddings and TFIDF. The highest accuracy (99.36%) was obtained by BiGRU with the document embeddings created using BART.

The TFIDF approach proves that the relevance of calculating the importance of each word from a document is an important factor for the fake news detection problem. This result is a direct consequence of the size of the document-term matrix used as the input. Moreover, the transformer embeddings obtained the best results among the document embeddings experiments as they manage to encode and preserve the context within the vector representation. When compared to the state-of-the-art model MisRoBÆRTa [23], the BiLSTM with BART obtained similar results, while the BiGRU with BART marginally outperformed the model with a 0.02% difference in accuracy. We hypothesize that this difference in performance is due to the use of pre-trained transformers instead of fine-tuned versions.

The recall metric is the most relevant one for the fake news detection task because it calculates the documents correctly classified as fake relative to all the actual fake documents, regardless of the predicted label. No clear pattern emerges among the document embeddings to determine which has the overall best performance. For example, when using LSTM, the best performance was obtained with document embedding DOCEMB WORD2VEC CBOW (96.84%), followed closely by DOCEMB FASTTEXT SG (96.72%), while, when using Multi-Layer Perceptron, the best performance was obtained by DOCEMB FASTTEXT SG (95.48%), followed closely by DOCEMB WORD2VEC SG (95.40%).

Finally, by analyzing the results, we observed the following:

(1) A simpler neural architecture offers similar or better results compared to complex deep learning architectures that employ multiple layers, i.e., in our comparison, we obtained similar results as the complex MisRoBÆRTa [23] architecture without fine-tuning the transformers;

(2) The embeddings used to vectorize the textual data make all the difference in performance, i.e., the right embedding must be selected to obtain good results with a given model;

(3) We need a data-driven approach to select the best model and the best embedding for our dataset.

4.4 Additional Experiments

In this section, we present more experiments using four additional datasets that are analyzed in detail in [68]. For this set of experiments, we compared our results with existing results from the current literature. Furthermore, we trained our own model for each dataset using MisRoBÆRTa [23], but we did not fine-tune the transformers. We used the pre-trained BART (*facebook/bart-large*) and ROBERTA (*roberta-base*) versions from HuggingFace [44]. We hypothesize that this is the reason we obtained similar results to the ones obtained with the models that use document embeddings with this state-of-the-art architecture.

Tables 3 and 4 present experimental results obtained on the LIAR dataset [10]. For our experiments, we used the dataset as it was initially released, with 6 labels [10] (Table 3), and by balancing the dataset's labels (Table 4) as proposed in [51]. To balance the labels, we created binary labels, i.e., all the texts that are not labeled with *true* are considered *false*. Using the same experimental configurations as presented in Section 4.2, we obtained results that are aligned with our original observations on the proposed dataset. Further, we obtained results similar to state-of-the-art results for the multi-label dataset, e.g., Wang [10] and Alhindi et al. [69] obtained an accuracy of ∼20%. For the binary classification, we obtained results that go beyond the the state of the art, e.g.,Upadhayay and Behzadan [51] obtains an accuracy of 70% while we obtain an accuracy of 83.99% with the LSTM model that employs the document embeddings constructed with GLOVE.

Table 3 presents the results obtained by the different machine and deep learning algorithms on the LIAR dataset [10]. The dataset contains approximately 12.8K human annotated short statements collected using POLITIFACT.COM's API. In this set of experiments, we used all the 6 labels of LIAR, i.e., *pants-fire*, *false*, *barely-true*, *half-true*, *mostly-true*, and *true*, to build our classification models. The dataset is highly imbalanced, as there are more news articles labeled with *true* than news articles labeled with the other five classes combined. Due to this high degree of imbalance, the models performed poorly. We observe that the best-performing models employ document embedding constructed with BART. The overall best performance model was Multi-Layer Perceptron with BART-built document embedding, with an accuracy of 25.89%. The overall difference between the worst- and best-performing models is approximately 7%. We note that for Naïve Bayes, the model trained with TFIDF obtained better scores than the models trained with document embedding. We observed no real difference in performance among the models trained with the document

Table 3: Fake news detection results on Liar dataset with 6 labels as presented in Wang [10]

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	23.07 ± 0.70	24.16 ± 2.03	23.07 ± 0.70	23.00 ± 0.93	22.86 ± 0.89	23.00 ± 0.93
DocEmb Word2Vec CBOW	18.32 ± 1.01	21.62 ± 1.42	18.32 ± 1.01	22.40 ± 0.59	22.37 ± 0.68	22.40 ± 0.59
DocEmb Word2Vec SG	20.42 ± 0.96	21.74 ± 0.84	20.42 ± 0.96	23.12 ± 0.69	23.29 ± 0.69	23.12 ± 0.69
DocEmb FastText CBOW	17.19 ± 0.63	21.89 ± 1.45	17.19 ± 0.63	22.68 ± 0.55	22.63 ± 0.71	22.68 ± 0.55
DocEmb FastText SG	19.85 ± 1.10	21.57 ± 1.22	19.85 ± 1.10	22.93 ± 0.83	23.00 ± 0.80	22.93 ± 0.83
DocEmb GloVe	17.60 ± 0.77	21.31 ± 1.18	17.60 ± 0.77	21.99 ± 0.63	21.72 ± 0.71	21.99 ± 0.63
DocEmb BERT	20.58 ± 0.71	22.40 ± 1.17	20.58 ± 0.71	23.78 ± 0.82	24.03 ± 0.97	23.78 ± 0.82
DocEmb RoBERTa	15.91 ± 1.02	20.31 ± 1.38	15.91 ± 1.02	21.09 ± 1.08	20.51 ± 0.85	21.09 ± 1.08
DocEmb BART	21.79 ± 0.90	24.07 ± 1.09	21.79 ± 0.90	24.93 ± 0.74	25.26 ± 0.83	24.93 ± 0.74
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	23.71 ± 0.68	24.79 ± 1.26	23.71 ± 0.68	23.04 ± 0.66	23.08 ± 0.77	23.04 ± 0.66
DocEmb Word2Vec CBOW	22.83 ± 0.65	22.37 ± 0.66	22.83 ± 0.65	22.70 ± 0.91	22.56 ± 0.87	22.70 ± 0.91
DocEmb Word2Vec SG	23.46 ± 0.73	23.19 ± 0.68	23.46 ± 0.73	23.26 ± 0.65	23.15 ± 1.09	23.26 ± 0.65
DocEmb FastText CBOW	22.34 ± 0.49	21.63 ± 0.81	22.34 ± 0.49	22.72 ± 0.89	22.16 ± 1.19	22.72 ± 0.89
DocEmb FastText SG	23.62 ± 0.82	23.29 ± 1.08	23.62 ± 0.82	23.48 ± 0.92	23.47 ± 1.13	23.48 ± 0.92
DocEmb GloVe	23.64 ± 0.55	22.54 ± 0.97	23.64 ± 0.55	23.24 ± 0.71	21.94 ± 1.31	23.24 ± 0.71
DocEmb BERT	24.06 ± 1.03	24.33 ± 1.00	24.06 ± 1.03	23.58 ± 0.66	23.88 ± 0.84	23.58 ± 0.66
DocEmb RoBERTa	21.66 ± 1.59	21.01 ± 1.60	21.66 ± 1.59	22.82 ± 0.61	20.83 ± 1.22	22.82 ± 0.61
DocEmb BART	25.60 ± 0.41	25.84 ± 0.18	25.60 ± 0.41	$\mathbf{25.89 \pm 0.75}$	26.15 ± 0.72	25.89 ± 0.75
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	21.77 ± 0.50	21.77 ± 0.48	21.77 ± 0.50	21.62 ± 0.49	21.60 ± 0.48	21.62 ± 0.49
DocEmb Word2Vec CBOW	22.70 ± 0.95	22.53 ± 0.94	22.70 ± 0.95	22.65 ± 0.60	22.57 ± 0.62	22.65 ± 0.60
DocEmb Word2Vec SG	23.66 ± 0.99	23.46 ± 1.02	23.66 ± 0.99	23.51 ± 0.69	23.14 ± 0.84	23.51 ± 0.69
DocEmb FastText CBOW	22.40 ± 1.06	22.23 ± 1.07	22.40 ± 1.06	22.53 ± 0.85	22.49 ± 0.86	22.53 ± 0.85
DocEmb FastText SG	23.50 ± 0.76	23.24 ± 0.90	23.50 ± 0.76	23.45 ± 0.61	23.41 ± 0.89	23.45 ± 0.61
DocEmb GloVe	23.59 ± 0.46	22.90 ± 1.40	23.59 ± 0.46	23.08 ± 0.42	22.77 ± 0.88	23.08 ± 0.42
DocEmb BERT	23.21 ± 0.55	23.37 ± 0.52	23.21 ± 0.55	23.31 ± 0.70	23.25 ± 0.76	23.31 ± 0.70
DocEmb RoBERTa	22.94 ± 1.00	21.65 ± 1.08	22.94 ± 1.00	22.96 ± 0.61	19.77 ± 1.66	22.96 ± 0.61
DocEmb BART	25.02 ± 0.57	25.08 ± 0.59	25.02 ± 0.57	25.75 ± 0.61	25.78 ± 0.62	25.75 ± 0.61
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	21.62 ± 0.50	21.63 ± 0.47	21.62 ± 0.50	21.43 ± 0.44	21.44 ± 0.46	21.43 ± 0.44
DocEmb Word2Vec CBOW	22.36 ± 1.00	22.14 ± 0.94	22.36 ± 1.00	22.51 ± 0.76	22.46 ± 0.79	22.51 ± 0.76
DocEmb Word2Vec SG	23.47 ± 0.53	23.02 ± 0.72	23.47 ± 0.53	23.47 ± 0.71	23.16 ± 0.66	23.47 ± 0.71
DocEmb FastText CBOW	22.62 ± 0.77	22.48 ± 0.72	22.62 ± 0.77	22.51 ± 0.53	22.42 ± 0.44	22.51 ± 0.53
DocEmb FastText SG	23.34 ± 0.55	23.08 ± 0.76	23.34 ± 0.55	23.54 ± 0.71	23.28 ± 0.58	23.54 ± 0.71
DocEmb GloVe	23.47 ± 0.64	22.84 ± 1.76	23.47 ± 0.64	23.21 ± 0.56	22.93 ± 1.27	23.21 ± 0.56
DocEmb BERT	23.64 ± 0.36	23.90 ± 0.53	23.64 ± 0.36	23.00 ± 0.72	23.21 ± 0.88	23.00 ± 0.72
DocEmb RoBERTa	22.69 ± 0.68	19.84 ± 1.95	22.69 ± 0.68	22.73 ± 0.72	21.55 ± 2.26	22.73 ± 0.72
DocEmb BART	24.99 ± 0.66	25.00 ± 0.69	24.99 ± 0.66	25.20 ± 0.88	25.26 ± 0.86	25.20 ± 0.88
	Accuracy		**Precision**		**Recall**	
MisRoBÆRTa [23]	24.62 ± 0.39		25.87 ± 0.67		24.61 ± 0.39	
	F1-Score					
Hybrid CNNs [10]	27.70					
BiLSTM [69]	26.00					

embeddings using word embeddings. This low performance is also present in the current literature [10, 69], with accuracy scores very similar to the ones obtained by the models we trained.

To mitigate the poor performance obtained using all 6 labels of the LIAR dataset and to minimize the imbalance between the classes, we employed a binarization approach to the dataset. This approach is also used in the current literature. For example, Upadhayay and Behzadan [51] and Yang et al. [29] also use the LIAR dataset with 2 labels, i.e., *true* and *false*, to train their models. On this dataset, we observed that the performance of all the models improved. Naïve Bayes trained on document embeddings obtained the worst results. The overall best results were obtained by LSTM with the document embedding constructed with GloVe, with an accuracy score of 83.99%. Furthermore, Naïve Bayes, Gradient Boosted Trees, and Perceptron obtained better results with the TFIDF vectorization. The performance of these models is directly impacted by TFIDF's features. The proposed approach outperforms more

Table 4: Fake news detection results on Liar dataset with 2 labels as presented in Upadhayay and Behzadan [51]

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	83.92 ± 0.04	83.96 ± 0.01	99.94 ± 0.04	83.64 ± 0.20	84.09 ± 0.08	99.31 ± 0.19
DocEmb Word2Vec CBOW	67.02 ± 1.21	86.09 ± 0.43	72.43 ± 1.76	83.28 ± 0.19	84.15 ± 0.07	98.68 ± 0.25
DocEmb Word2Vec SG	64.57 ± 3.54	86.07 ± 0.31	68.96 ± 5.01	83.30 ± 0.33	84.15 ± 0.11	98.70 ± 0.35
DocEmb FastText CBOW	67.89 ± 2.19	85.60 ± 0.32	74.26 ± 3.20	83.25 ± 0.28	84.16 ± 0.12	98.61 ± 0.29
DocEmb FastText SG	65.19 ± 4.32	86.17 ± 0.43	69.75 ± 6.17	83.28 ± 0.23	84.16 ± 0.10	98.65 ± 0.32
DocEmb GloVe	59.71 ± 1.80	85.89 ± 0.52	62.24 ± 2.54	83.05 ± 0.26	84.10 ± 0.08	98.42 ± 0.29
DocEmb BERT	61.21 ± 0.91	86.76 ± 0.58	63.49 ± 1.14	83.38 ± 0.19	84.16 ± 0.09	98.80 ± 0.19
DocEmb RoBERTa	60.04 ± 3.98	85.30 ± 0.53	63.35 ± 6.07	83.36 ± 0.20	84.02 ± 0.08	99.01 ± 0.25
DocEmb BART	62.11 ± 1.03	87.65 ± 0.52	63.87 ± 1.25	83.46 ± 0.19	84.36 ± 0.12	98.58 ± 0.32
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01	80.87 ± 0.69	84.50 ± 0.15	94.58 ± 1.13
DocEmb Word2Vec CBOW	83.88 ± 0.06	83.96 ± 0.02	99.88 ± 0.06	83.96 ± 0.04	83.98 ± 0.02	99.97 ± 0.03
DocEmb Word2Vec SG	83.94 ± 0.04	83.97 ± 0.01	99.95 ± 0.06	83.90 ± 0.10	83.99 ± 0.04	99.86 ± 0.10
DocEmb FastText CBOW	83.87 ± 0.06	83.98 ± 0.03	99.82 ± 0.08	83.95 ± 0.06	83.99 ± 0.03	99.94 ± 0.07
DocEmb FastText SG	83.97 ± 0.02	83.97 ± 0.01	99.99 ± 0.01	83.93 ± 0.08	83.99 ± 0.02	99.91 ± 0.10
DocEmb GloVe	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01	83.96 ± 0.01	83.97 ± 0.01	99.99 ± 0.01
DocEmb BERT	83.81 ± 0.11	83.98 ± 0.05	99.74 ± 0.14	83.18 ± 0.52	84.25 ± 0.25	98.37 ± 1.11
DocEmb RoBERTa	83.96 ± 0.03	83.97 ± 0.01	99.99 ± 0.03	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01
DocEmb BART	83.44 ± 0.55	84.37 ± 0.28	98.53 ± 1.17	81.63 ± 1.01	84.97 ± 0.33	94.91 ± 1.52
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	77.05 ± 0.89	84.75 ± 0.20	88.62 ± 1.13	76.96 ± 0.86	84.75 ± 0.17	88.49 ± 1.08
DocEmb Word2Vec CBOW	81.43 ± 0.57	84.44 ± 0.13	95.47 ± 0.88	80.51 ± 1.02	84.63 ± 0.18	93.83 ± 1.65
DocEmb Word2Vec SG	83.86 ± 0.13	84.04 ± 0.05	99.72 ± 0.13	83.79 ± 0.16	84.07 ± 0.05	99.57 ± 0.17
DocEmb FastText CBOW	81.20 ± 0.74	84.45 ± 0.23	95.13 ± 1.03	80.77 ± 1.12	84.49 ± 0.34	94.44 ± 1.63
DocEmb FastText SG	83.88 ± 0.14	84.01 ± 0.04	99.79 ± 0.16	83.78 ± 0.19	84.00 ± 0.06	99.66 ± 0.19
DocEmb GloVe	**83.99 ± 0.02**	83.98 ± 0.02	99.99 ± 0.01	83.98 ± 0.04	83.98 ± 0.02	99.99 ± 0.01
DocEmb BERT	80.37 ± 1.53	84.84 ± 0.49	93.31 ± 2.74	79.69 ± 1.76	85.05 ± 0.35	92.00 ± 2.95
DocEmb RoBERTa	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01
DocEmb BART	81.41 ± 0.85	84.93 ± 0.22	94.66 ± 1.21	81.43 ± 0.94	85.14 ± 0.22	94.34 ± 1.35
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	76.99 ± 0.59	84.71 ± 0.19	88.58 ± 0.65	76.87 ± 0.66	84.72 ± 0.22	88.39 ± 0.94
DocEmb Word2Vec CBOW	81.52 ± 0.67	84.49 ± 0.24	95.54 ± 0.95	80.52 ± 1.01	84.58 ± 0.15	93.92 ± 1.53
DocEmb Word2Vec SG	83.91 ± 0.13	84.04 ± 0.04	99.80 ± 0.14	83.81 ± 0.16	84.06 ± 0.05	99.60 ± 0.18
DocEmb FastText CBOW	80.93 ± 1.14	84.44 ± 0.24	94.74 ± 1.93	80.20 ± 0.96	84.67 ± 0.27	93.31 ± 1.55
DocEmb FastText SG	83.89 ± 0.12	84.01 ± 0.04	99.80 ± 0.13	83.82 ± 0.16	84.02 ± 0.05	99.70 ± 0.16
DocEmb GloVe	83.98 ± 0.03	83.98 ± 0.02	99.99 ± 0.01	83.98 ± 0.03	83.99 ± 0.02	99.99 ± 0.01
DocEmb BERT	79.86 ± 2.30	84.73 ± 0.40	92.75 ± 3.83	79.94 ± 1.25	84.93 ± 0.36	92.54 ± 2.08
DocEmb RoBERTa	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01	83.97 ± 0.01	83.97 ± 0.01	99.99 ± 0.01
DocEmb BART	81.22 ± 0.75	85.13 ± 0.27	94.06 ± 1.19	80.60 ± 0.90	85.24 ± 0.22	93.01 ± 1.34
	Accuracy		**Precision**		**Recall**	
MisRoBÆRTa [23]	81.15 ± 0.07		81.15 ± 0.07		81.16 ± 0.07	
					Accuracy	
CNN with BERT-base embeddings [51]					70.00	
UFD [29]					75.90	

complex models proposed in the current literature, e.g., CNN with BERT-base embeddings [51] obtained an accuracy of 70% and UFD [29] obtained an accuracy of 75.90%.

Tables 5–7 present the experimental results obtained on the Kaggle [12, 18], Buzz Feed News [52], and TSHP-17 datasets as presented in [53, 54]. Both Kaggle and Buzz Feed News are binary datasets, i.e., with the levels reliable and false. To emphasize that the embedding makes the main difference and that the models can generalize when we move from binary to multi-class classification, we used the multilabel dataset TSHP-17, which has the following 3 classes: satire, hoax, and propaganda. For this set of experiments, we used the same experimental setup and algorithm configurations as presented above. Again, we obtained results that are aligned with our original observations, reinforcing our claims.

Table 5: Fake news detection results on the Kaggle dataset as presented in Kaliyar et al. [12, 18]

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	89.54 ± 0.20	92.94 ± 0.33	84.93 ± 0.67	96.74 ± 0.15	96.22 ± 0.36	97.10 ± 0.29
DocEmb Word2Vec CBOW	71.14 ± 0.41	78.89 ± 0.63	55.41 ± 0.62	91.90 ± 0.23	92.01 ± 0.46	91.24 ± 0.40
DocEmb Word2Vec SG	60.91 ± 0.53	85.19 ± 1.69	23.66 ± 1.16	93.31 ± 0.37	93.81 ± 0.32	92.33 ± 0.55
DocEmb FastText CBOW	67.45 ± 0.67	77.26 ± 1.05	46.75 ± 1.06	91.45 ± 0.29	91.32 ± 0.68	91.06 ± 0.49
DocEmb FastText SG	60.22 ± 0.22	88.13 ± 1.08	20.93 ± 0.41	93.41 ± 0.31	94.05 ± 0.34	92.28 ± 0.58
DocEmb GloVe	62.22 ± 0.49	81.02 ± 1.27	29.04 ± 0.91	90.63 ± 0.30	89.98 ± 0.48	90.83 ± 0.36
DocEmb BERT	70.52 ± 0.47	82.03 ± 0.70	50.34 ± 1.06	92.91 ± 0.34	93.58 ± 0.22	91.69 ± 0.65
DocEmb RoBERTa	81.86 ± 0.56	87.45 ± 0.78	73.18 ± 1.27	92.42 ± 0.22	92.51 ± 0.33	91.82 ± 0.45
DocEmb BART	90.14 ± 0.42	91.69 ± 0.63	87.64 ± 0.52	99.06 ± 0.11	98.82 ± 0.24	99.24 ± 0.19
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	93.76 ± 0.27	94.45 ± 0.51	92.59 ± 0.56	95.36 ± 0.23	95.24 ± 0.42	95.20 ± 0.44
DocEmb Word2Vec CBOW	90.15 ± 0.35	90.84 ± 0.63	88.67 ± 0.71	91.54 ± 0.40	91.67 ± 1.42	90.89 ± 1.45
DocEmb Word2Vec SG	88.98 ± 0.46	90.48 ± 0.51	86.41 ± 0.86	92.45 ± 0.32	92.59 ± 1.19	91.83 ± 1.29
DocEmb FastText CBOW	90.10 ± 0.44	90.23 ± 0.95	89.30 ± 0.62	92.00 ± 0.44	91.99 ± 1.03	91.54 ± 1.68
DocEmb FastText SG	88.45 ± 0.59	90.65 ± 0.82	84.99 ± 1.07	92.15 ± 0.43	92.78 ± 1.03	90.94 ± 0.81
DocEmb GloVe	83.90 ± 0.52	85.26 ± 1.15	80.89 ± 1.97	87.57 ± 0.57	88.79 ± 2.16	85.29 ± 2.64
DocEmb BERT	92.09 ± 0.50	92.84 ± 1.01	90.74 ± 1.36	94.80 ± 0.47	94.51 ± 1.95	94.89 ± 1.73
DocEmb RoBERTa	91.62 ± 0.40	91.02 ± 2.03	91.92 ± 1.94	92.74 ± 0.96	93.03 ± 3.76	92.31 ± 4.31
DocEmb BART	99.73 ± 0.09	99.66 ± 0.11	99.78 ± 0.13	99.77 ± 0.07	99.70 ± 0.14	99.82 ± 0.08
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	95.05 ± 0.25	94.83 ± 0.44	95.00 ± 0.50	95.01 ± 0.25	94.75 ± 0.39	95.00 ± 0.34
DocEmb Word2Vec CBOW	93.70 ± 0.32	94.11 ± 1.11	92.87 ± 1.50	93.81 ± 0.39	93.60 ± 0.82	93.68 ± 0.87
DocEmb Word2Vec SG	93.03 ± 0.32	93.22 ± 1.35	92.40 ± 1.24	93.14 ± 0.50	94.08 ± 1.74	91.71 ± 1.80
DocEmb FastText CBOW	93.25 ± 0.65	92.83 ± 2.21	93.44 ± 2.16	93.52 ± 0.30	93.04 ± 1.66	93.73 ± 2.08
DocEmb FastText SG	92.90 ± 0.41	93.13 ± 1.17	92.21 ± 1.15	92.67 ± 0.56	94.61 ± 1.65	90.12 ± 2.34
DocEmb GloVe	88.84 ± 0.82	88.36 ± 2.96	88.98 ± 2.98	88.83 ± 0.79	89.23 ± 3.27	87.93 ± 4.70
DocEmb BERT	96.31 ± 0.72	96.15 ± 2.41	96.36 ± 1.84	96.36 ± 1.32	96.08 ± 3.10	96.59 ± 1.13
DocEmb RoBERTa	93.52 ± 0.78	94.41 ± 2.02	92.24 ± 2.94	93.12 ± 1.29	94.87 ± 2.53	90.93 ± 4.62
DocEmb BART	99.79 ± 0.06	99.74 ± 0.11	99.84 ± 0.08	$\mathbf{99.80 \pm 0.12}$	99.72 ± 0.18	99.87 ± 0.11
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	94.99 ± 0.22	94.64 ± 0.34	95.08 ± 0.45	94.98 ± 0.27	94.71 ± 0.38	94.98 ± 0.38
DocEmb Word2Vec CBOW	93.67 ± 0.41	93.66 ± 1.02	93.32 ± 1.19	93.68 ± 0.30	93.48 ± 0.67	93.54 ± 1.20
DocEmb Word2Vec SG	93.05 ± 0.36	93.28 ± 1.48	92.40 ± 1.35	92.91 ± 0.77	93.55 ± 2.49	91.87 ± 2.32
DocEmb FastText CBOW	93.18 ± 0.54	92.94 ± 2.21	93.14 ± 1.85	93.34 ± 0.59	92.43 ± 1.26	94.02 ± 1.01
DocEmb FastText SG	92.73 ± 0.76	92.94 ± 2.54	92.16 ± 2.16	92.86 ± 0.45	94.07 ± 1.14	91.09 ± 2.10
DocEmb GloVe	88.83 ± 0.76	89.73 ± 3.25	87.26 ± 3.54	89.19 ± 0.64	90.81 ± 1.47	86.59 ± 2.85
DocEmb BERT	96.25 ± 0.52	96.81 ± 1.95	95.50 ± 1.87	96.37 ± 0.81	97.71 ± 0.90	94.77 ± 2.28
DocEmb RoBERTa	92.32 ± 1.93	92.10 ± 5.31	92.80 ± 5.35	92.93 ± 1.27	95.36 ± 2.48	90.01 ± 4.51
DocEmb BART	99.77 ± 0.07	99.72 ± 0.14	99.82 ± 0.09	99.78 ± 0.07	99.72 ± 0.12	99.83 ± 0.09
				Accuracy	**Precision**	**Recall**
MisRoBÆRTa [23]				97.57 ± 0.29	97.58 ± 0.28	97.57 ± 0.31
C-CNN [24]				99.90	99.90	99.90
				Accuracy		
FNDNet [12]				98.36		
FakeBERT [18]				98.90		

Table 5 presents the results obtained on the Kaggle dataset [12, 18]. We observed that only for the Gradient Boosted Trees and Multi-Layer Perceptron models, the document embeddings obtained with Word2Vec SG and FastText SG outperformed their CBOW counterparts. When analyzing the same document embedding, i.e., DocEmb, we observed very little difference in performance among the neural models. As we used early stopping mechanisms, the neural network models did not overfit. Among the document embeddings employing transformers, the ones that use BART obtained the best results across all experiments. With an accuracy of 99.80%, the overall best-performing model is the Bidirectional LSTM with document embeddings constructed with BART, i.e., DocEmb BART. The results show that our approach outperforms more complex models proposed in the current literature, e.g., FNDNet [12] obtained an accuracy of 98.36%, FakeBERT [18] obtained an accuracy of 98.90%, and C-CNN [24] obtained an accuracy of 99.90%. We observe that C-CNN, a large neural model with multiple layers that also concatenates the

Table 6: Fake news detection results on the Buzz Feed News dataset as presented in Horne and Adali [52]

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	78.29 ± 0.99	70.70 ± 2.62	78.29 ± 0.99
DocEmb Word2Vec CBOW	55.26 ± 1.52	72.29 ± 2.16	55.26 ± 1.52	78.72 ± 0.88	72.97 ± 2.71	78.72 ± 0.88
DocEmb Word2Vec SG	54.42 ± 2.99	72.24 ± 2.05	54.42 ± 2.99	78.97 ± 0.91	73.30 ± 2.66	78.97 ± 0.91
DocEmb FastText CBOW	49.13 ± 2.26	71.76 ± 1.86	49.13 ± 2.26	78.54 ± 0.57	72.26 ± 1.50	78.54 ± 0.57
DocEmb FastText SG	55.58 ± 2.35	73.95 ± 1.76	55.58 ± 2.35	78.94 ± 0.66	73.46 ± 2.97	78.94 ± 0.66
DocEmb GloVe	48.63 ± 8.40	69.79 ± 3.13	48.63 ± 8.40	78.16 ± 1.35	71.89 ± 4.01	78.16 ± 1.35
DocEmb BERT	59.91 ± 1.79	76.22 ± 0.85	59.91 ± 1.79	78.44 ± 0.73	71.31 ± 1.95	78.44 ± 0.73
DocEmb RoBERTa	62.02 ± 7.65	70.54 ± 1.52	62.02 ± 7.65	77.98 ± 0.70	69.83 ± 4.17	77.98 ± 0.70
DocEmb BART	61.56 ± 1.29	80.57 ± 1.17	61.56 ± 1.29	79.28 ± 1.18	73.92 ± 2.50	79.28 ± 1.18
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	78.29 ± 0.50	70.91 ± 6.32	78.29 ± 0.50
DocEmb Word2Vec CBOW	77.98 ± 0.31	63.63 ± 3.26	77.98 ± 0.31	78.10 ± 0.50	65.94 ± 5.52	78.10 ± 0.50
DocEmb Word2Vec SG	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb FastText CBOW	77.79 ± 0.62	66.38 ± 5.91	77.79 ± 0.62	77.91 ± 0.65	66.63 ± 3.36	77.91 ± 0.65
DocEmb FastText SG	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb GloVe	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb BERT	77.76 ± 1.02	68.20 ± 3.39	77.76 ± 1.02	77.60 ± 1.65	70.76 ± 3.70	77.60 ± 1.65
DocEmb RoBERTa	77.85 ± 0.33	63.79 ± 5.23	77.85 ± 0.33	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb BART	$\mathbf{79.78 \pm 0.84}$	75.84 ± 1.30	79.78 ± 0.84	79.75 ± 1.75	75.40 ± 2.30	79.75 ± 1.75
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	78.91 ± 0.98	73.90 ± 1.40	78.91 ± 0.98	78.63 ± 0.80	73.77 ± 1.63	78.63 ± 0.80
DocEmb Word2Vec CBOW	77.88 ± 1.40	70.88 ± 3.53	77.88 ± 1.40	77.23 ± 1.55	70.58 ± 2.30	77.23 ± 1.55
DocEmb Word2Vec SG	78.04 ± 0.16	63.35 ± 1.97	78.04 ± 0.16	77.88 ± 0.59	67.35 ± 2.43	77.88 ± 0.59
DocEmb FastText CBOW	78.07 ± 0.59	71.74 ± 2.23	78.07 ± 0.59	78.10 ± 0.79	73.05 ± 1.58	78.10 ± 0.79
DocEmb FastText SG	77.98 ± 0.20	61.64 ± 1.69	77.98 ± 0.20	77.85 ± 0.74	67.51 ± 5.94	77.85 ± 0.74
DocEmb GloVe	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.85 ± 0.17	62.74 ± 3.44	77.85 ± 0.17
DocEmb BERT	77.57 ± 2.95	73.20 ± 2.13	77.57 ± 2.95	77.51 ± 2.22	74.46 ± 2.19	77.51 ± 2.22
DocEmb RoBERTa	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb BART	78.07 ± 2.16	76.31 ± 3.01	78.07 ± 2.16	77.35 ± 2.39	75.97 ± 1.89	77.35 ± 2.39
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	78.94 ± 1.21	73.99 ± 2.19	78.94 ± 1.21	78.60 ± 1.13	73.69 ± 1.40	78.60 ± 1.13
DocEmb Word2Vec CBOW	77.57 ± 1.26	70.72 ± 2.80	77.57 ± 1.26	77.32 ± 1.31	71.04 ± 2.30	77.32 ± 1.31
DocEmb Word2Vec SG	78.29 ± 0.37	66.63 ± 2.52	78.29 ± 0.37	78.22 ± 1.00	68.24 ± 2.56	78.22 ± 1.00
DocEmb FastText CBOW	78.19 ± 0.78	72.06 ± 2.37	78.19 ± 0.78	77.73 ± 1.51	74.22 ± 1.06	77.73 ± 1.51
DocEmb FastText SG	77.88 ± 0.37	64.47 ± 3.68	77.88 ± 0.37	77.76 ± 0.79	67.54 ± 2.82	77.76 ± 0.79
DocEmb GloVe	77.82 ± 0.31	61.81 ± 2.43	77.82 ± 0.31	77.66 ± 0.52	64.45 ± 3.17	77.66 ± 0.52
DocEmb BERT	78.54 ± 1.39	72.23 ± 3.39	78.54 ± 1.39	74.86 ± 4.28	72.99 ± 2.21	74.86 ± 4.28
DocEmb RoBERTa	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01	77.88 ± 0.01	60.66 ± 0.01	77.88 ± 0.01
DocEmb BART	77.48 ± 2.08	75.96 ± 2.38	77.48 ± 2.08	76.45 ± 4.56	77.31 ± 3.14	76.45 ± 4.56
	Accuracy		**Precision**		**Recall**	
MisRoBÆRTa [23]	77.39 ± 0.83		77.39 ± 0.83		77.39 ± 0.83	
	Accuracy					
SVM [52]	78.00					
UFD [29]	67.90					

results of three CNN models, outperforms the Bidirectional LSTM in terms of average accuracy on the Kaggle dataset by only 0.10%. We also want to emphasize that the results in Table 5 present the mean over 10 runs for each metric per model and embedding pair. Thus, if we only take the best-performing model as in the case of the C-CNN results presented by Sedik et al. [24], then the Bidirectional LSTM model manages to obtain an accuracy of $99.92\%(= 99.80\%$ (mean accuracy) $+0.12\%$ (standard deviation)).

Table 6 presents the results obtained on the Buzz Feed News dataset [52]. On this dataset, we observed that all the models obtained good results with TFIDF, such that some models that employ the TFIDF vectorization outperformed the document embeddings constructed with word and transformer embeddings, see, e.g., the results for LSTM, Bidirectional LSTM, GRU, and Bidirectional GRU. With an accuracy of 79.78%, the overall best-performing model is Perceptron with BART document embeddings. For all the models, there is very little difference between the document embeddings that employ CBOW and their Skip-Gram counterparts. The results show that our approach outperforms

Table 7: Fake news detection results on TSHP-17 dataset as presented in Rashkin et al. [53] and Barrón-Cedeño et al. [54]

	Naïve Bayes			Gradient Boosted Trees		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	92.08 ± 0.15	92.11 ± 0.15	92.08 ± 0.15	98.05 ± 0.12	98.05 ± 0.12	98.05 ± 0.12
DocEmb Word2Vec CBOW	70.06 ± 0.50	73.01 ± 0.36	70.06 ± 0.50	95.64 ± 0.26	95.63 ± 0.26	95.64 ± 0.26
DocEmb Word2Vec SG	55.33 ± 0.63	68.18 ± 0.33	55.33 ± 0.63	95.76 ± 0.21	95.75 ± 0.21	95.76 ± 0.21
DocEmb FastText CBOW	62.83 ± 0.47	70.17 ± 0.66	62.83 ± 0.47	94.36 ± 0.27	94.34 ± 0.27	94.36 ± 0.27
DocEmb FastText SG	59.72 ± 0.52	69.49 ± 0.63	59.72 ± 0.52	95.66 ± 0.28	95.65 ± 0.28	95.66 ± 0.28
DocEmb GloVe	52.99 ± 0.56	65.84 ± 0.43	52.99 ± 0.56	96.19 ± 0.28	96.19 ± 0.28	96.19 ± 0.28
DocEmb BERT	84.65 ± 0.63	87.60 ± 0.42	84.65 ± 0.63	98.18 ± 0.10	98.18 ± 0.10	98.18 ± 0.10
DocEmb RoBERTa	52.15 ± 0.33	65.77 ± 0.64	52.15 ± 0.33	79.15 ± 0.36	79.11 ± 0.38	79.15 ± 0.36
DocEmb BART	94.08 ± 0.28	94.66 ± 0.26	94.08 ± 0.28	99.01 ± 0.10	99.01 ± 0.10	99.01 ± 0.10
	Perceptron			**Multi Layer Perceptron**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	97.64 ± 0.13	97.64 ± 0.13	97.64 ± 0.13	97.54 ± 0.16	97.54 ± 0.16	97.54 ± 0.16
DocEmb Word2Vec CBOW	94.24 ± 0.23	94.22 ± 0.24	94.24 ± 0.23	96.11 ± 0.22	96.11 ± 0.22	96.11 ± 0.22
DocEmb Word2Vec SG	90.14 ± 0.29	90.09 ± 0.29	90.14 ± 0.29	93.91 ± 0.24	93.89 ± 0.24	93.91 ± 0.24
DocEmb FastText CBOW	93.14 ± 0.30	93.14 ± 0.30	93.14 ± 0.30	95.63 ± 0.24	95.64 ± 0.24	95.63 ± 0.24
DocEmb FastText SG	90.00 ± 0.20	89.94 ± 0.21	90.00 ± 0.20	93.64 ± 0.30	93.64 ± 0.29	93.64 ± 0.30
DocEmb GloVe	90.97 ± 0.27	90.95 ± 0.27	90.97 ± 0.27	94.04 ± 0.30	94.05 ± 0.29	94.04 ± 0.30
DocEmb BERT	98.44 ± 0.15	98.44 ± 0.15	98.44 ± 0.15	98.78 ± 0.14	98.78 ± 0.13	98.78 ± 0.14
DocEmb RoBERTa	77.79 ± 1.85	78.89 ± 0.65	77.79 ± 1.85	80.30 ± 1.55	81.29 ± 0.61	80.30 ± 1.55
DocEmb BART	99.54 ± 0.05	99.54 ± 0.05	99.54 ± 0.05	99.55 ± 0.06	99.55 ± 0.06	99.55 ± 0.06
	LSTM			**Bidirectional LSTM**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	97.10 ± 0.17	97.10 ± 0.17	97.10 ± 0.17	97.03 ± 0.22	97.03 ± 0.22	97.03 ± 0.22
DocEmb Word2Vec CBOW	96.93 ± 0.16	96.94 ± 0.15	96.93 ± 0.16	96.88 ± 0.09	96.88 ± 0.09	96.88 ± 0.09
DocEmb Word2Vec SG	95.37 ± 0.17	95.40 ± 0.17	95.37 ± 0.17	95.55 ± 0.32	95.56 ± 0.30	95.55 ± 0.32
DocEmb FastText CBOW	96.24 ± 0.30	96.26 ± 0.29	96.24 ± 0.30	96.37 ± 0.23	96.38 ± 0.23	96.37 ± 0.23
DocEmb FastText SG	95.06 ± 0.13	95.07 ± 0.13	95.06 ± 0.13	95.10 ± 0.37	95.11 ± 0.32	95.10 ± 0.37
DocEmb GloVe	95.17 ± 0.34	95.22 ± 0.28	95.17 ± 0.34	95.31 ± 0.42	95.35 ± 0.36	95.31 ± 0.42
DocEmb BERT	98.86 ± 0.24	98.87 ± 0.22	98.86 ± 0.24	98.89 ± 0.17	98.89 ± 0.16	98.89 ± 0.17
DocEmb RoBERTa	80.25 ± 1.38	81.31 ± 0.73	80.25 ± 1.38	80.17 ± 1.66	81.30 ± 0.86	80.17 ± 1.66
DocEmb BART	99.62 ± 0.05	99.62 ± 0.05	99.62 ± 0.05	$\mathbf{99.65 \pm 0.05}$	99.65 ± 0.05	99.65 ± 0.05
	GRU			**Bidirectional GRU**		
Vectorization	**Accuracy**	**Precision**	**Recall**	**Accuracy**	**Precision**	**Recall**
TFIDF	97.00 ± 0.13	97.00 ± 0.13	97.00 ± 0.13	96.85 ± 0.20	96.86 ± 0.20	96.85 ± 0.20
DocEmb Word2Vec CBOW	96.85 ± 0.16	96.86 ± 0.16	96.85 ± 0.16	96.86 ± 0.14	96.87 ± 0.14	96.86 ± 0.14
DocEmb Word2Vec SG	95.29 ± 0.28	95.31 ± 0.26	95.29 ± 0.28	95.53 ± 0.14	95.56 ± 0.14	95.53 ± 0.14
DocEmb FastText CBOW	96.28 ± 0.17	96.29 ± 0.17	96.28 ± 0.17	96.25 ± 0.28	96.26 ± 0.26	96.25 ± 0.28
DocEmb FastText SG	94.72 ± 0.31	94.75 ± 0.28	94.72 ± 0.31	94.77 ± 0.52	94.85 ± 0.44	94.77 ± 0.52
DocEmb GloVe	95.06 ± 0.34	95.10 ± 0.31	95.06 ± 0.34	95.09 ± 0.29	95.15 ± 0.25	95.09 ± 0.29
DocEmb BERT	98.91 ± 0.27	98.92 ± 0.25	98.91 ± 0.27	98.99 ± 0.10	98.99 ± 0.10	98.99 ± 0.10
DocEmb RoBERTa	80.44 ± 1.26	81.44 ± 0.55	80.44 ± 1.26	80.04 ± 1.69	81.36 ± 0.76	80.04 ± 1.69
DocEmb BART	99.62 ± 0.09	99.62 ± 0.09	99.62 ± 0.09	99.64 ± 0.07	99.64 ± 0.07	99.64 ± 0.07
	Accuracy		**Precision**		**Recall**	
MisRoBÆRTa [23]	99.52 ± 0.12		99.52 ± 0.12		99.52 ± 0.12	
	Accuracy					
Proppy [70]	98.36					

more complex models proposed in the current literature, e.g., SVM [52] obtained an accuracy of 78.00% and UFD [29] obtained an accuracy of 67.90%.

Table 7 presents our final experiments, on the TSHP-17 dataset [53, 54]. We observed that the document embeddings (DocEmbs) obtained with Word2Vec SG and FastText SG outperformed their CBOW counterparts only for the Gradient Boosted Trees model. All the models that employ TFIDF outperformed their counterparts that employ document embedding built with word embeddings. Among the document embeddings employing a transformer, the ones built with BART obtained the best results with regards to the model, while the ones that employ RoBERTa obtained the worst results. With an accuracy of 99.65%, the overall best-performing model is the Bidirectional LSTM with BART document embeddings. For the same DocEmb, we observed very little difference in performance among the neural models. The results show that our approach outperforms more complex models proposed in the current literature, e.g., Proppy's [70] accuracy is 98.36%.

Thus, in conclusion, we show, on five additional datasets, that:

(1) A simpler neural architecture offers at least similar or better results as complex architectures that employ multiple layers, and

(2) The difference in performance lies in the embeddings used to vectorize the textual data.

Furthermore, we generally obtained better results than in other current state-of-the-art work:

(1) On the LIAR dataset with 6 labels, Wang [10] obtained an F_1-Score of 27.7% using Hybrid CNNs and Alhindi et al. [69] obtained an F_1-Score of 26% using BiLSTM, while we obtained an accuracy of 25.89% using Multi-Layer Perceptron with the document embeddings employing BART;

(2) On the LIAR dataset with 2 labels, Upadhayay and Behzadan [51] obtained an accuracy of 70% using CNN with BERT-base embeddings, while we obtained an accuracy of 83.99% using LSTM with the document embeddings employing GLOVE;

(3) On the Kaggle dataset, the large deep learning model FakeBERT [18] obtained an accuracy of 98.90% and C-CNN [24] obtained an accuracy of 99.90%, while we obtained an accuracy of 99.80% using a simple Bidirectional LSTM with the document embeddings employing BART;

(4) On the Buzz Feed News dataset, Horne and Adali [52] obtained an accuracy of 78% using a linear SVM, while we obtained an accuracy of 79.78% using Perceptron with the document embeddings employing BART;

(5) On the TSHP-17 dataset, Barrón-Cedeño et al. [54] obtained an accuracy of 97.58% using Proppy [70], while we obtained 99.65% using Bidirectional LSTM with the document embeddings employing BART;

To sum up, this set of experiments again enforces our observations that the embedding is more important than the complexity of the classification architecture. Furthermore, there is no generic model that offers the best performance regardless of the dataset. Thus, a data-driven approach together with hyper-parameter tuning and ablation testing should be considered when the goal is to determine the best-performing model for a given dataset.

5 Discussion

Word embeddings manage to capture both local and global contexts as defined in Truică et al. [71]. These help the machine learning algorithms to model and learn the text context, syntax, and semantics, but fail to differentiate among the words' grammatical functions, i.e., the same word embedding is computed for a word regardless of its part-of-speech. On the other hand, transformer embeddings manage to learn the linguistic meaning of words, as they manage to preserve context by design. Thus, the same word has a different embedding depending on its lexical sense and concept as well as part-of-speech. Based on this, we can observe that the experiments that use document embeddings that employ transformers perform better than those that employ word embedding on average. The most interesting results, however, are those obtained with the document representation obtained with TFIDF. We observed that only the frequency-based importance of a word to a document within a textual corpus has a high impact on the models' performance. As a general observation, we observed very little difference in performance among the neural models when using the same document embedding.

The experimental results show that the DOCEMBs that use WORD2VEC and FASTTEXT obtain very similar results, with a difference of $\sim\pm 2\%$ when using Perceptron, and $\sim\pm 0.5\%$ when using LSTM. GLOVE based DOCEMB obtains the best results together with the LSTM model on the LIAR dataset when using 2 labels. For the sample extracted from the Fake News Corpus as well as the LIAR with 6 labels, Kaggle, and TSHP-17 datasets, the BART DOCEMB obtains the best results with different classification algorithms. We can conclude that there is no clear result for a classification model that generalizes well regardless of the dataset.

From the experimental results, we could not determine a clear winner with regards to document embedding and classification model. We observed empirically that the best-performing classification model changes with the dataset and the document embedding employed.

Our DOCEMB solutions were compared to the results we obtained when employing MisRoBÆRTa [23], a more complex state-of-the-art model that employs fine-tune BART and ROBERTA embeddings. We note that we did not use fine-tuning for our dataset as the authors did in the original work [23]. Thus, we used the pre-trained BART (*facebook/bart-large*) and ROBERTA (*roberta-base*) from HuggingFage [44]. Furthermore, we also compared the results we obtained on each dataset with the results obtained with other state-of-the-art models presented in the current literature.

In our experiments, we obtained results that lead to the following observation: feature selection is more important than the Deep Learning Architecture used for classification. To put it bluntly, the need to stack layers upon layers of neural cells, just to claim a novel architecture, does not solve real-world problems, it just exacerbates out of proportion our understanding of how to use Machine Learning/Deep Learning for Natural Language Processing tasks. To conclude our findings:

(1) A simpler neural architecture offers similar if not better results as complex deep learning architectures that employ multiple layers, i.e., in our comparison, we obtained similar results as the complex MisRoBÆRTa [23] architecture, better than state-of-the-art results, i.e., FakeBERT [18], and Poppy [70];

(2) The embeddings used to vectorize the textual data makes all the difference in performance, i.e., the right embedding must be selected to obtain good results with a given model;

(3) We need a data-driven approach to select the best model and the best embedding for our dataset;

(4) The way the word embedding manages to encapsulate the semantic, syntactic, and context features improves the performance of the classification models.

6 Conclusions

In this article, we presented a new approach for fake news detection using document embeddings (DOCEMBs). We also proposed a benchmark to establish the most efficient ways for finding misleading information. To detect fake news, we used multiple machine learning algorithms together with DOCEMBs built using either TFIDF, or word and transformer embeddings: WORD2VEC SG and CBOW, FASTTEXT SG and CBOW, GLOVE, BERT, ROBERTA, and BART.

Our approach emphasizes the importance of an overall document representation when dealing with the task of fake news detection and shows state-of-the-art performance results. Depending on the dataset, the results show that BIGRU/BILSTM with DOCEMB BART outperforms the other models. In the experiments, we obtained better results than state-of-the-art Deep Neural Network models, even though we used a simpler Deep Neural Network Architecture. Additionally, we obtained similar results as MisRoBÆRTa [23] when using pre-trained BART (*facebook/bart-large*) and ROBERTA (*roberta-base*) from HuggingFace [44]. These are significant results, not because of the evaluation scores but because of the complexity of the models. The main takeaway of this work is that a simpler neural architecture offers similar if not better results as complex architectures that employ multiple layers. We observe that the most relevant factor is the embedding employed for classification, as it can really make a difference.

In future research, we plan to use sentiment analysis with fake news detection to determine if there is a correlation between polarity and veracity. We also aim to use ensemble models that combine our proposed method with existing methods to determine if the performance of fake news detection is improved.

Acknowledgement: This work was done within the "AI-based conversational agent for misinformation fact-checking" project financed through the OPTIM Research framework (POCU grant no. 62461/03.06.2022, SMIS code 153735) and partially funded by the University Politehnica of Bucharest through the PubArt program.

References

[1] Truică, C.O.; APOSTOL, E.S.; Ștefu, T.; Karras, P. A Deep Learning Architecture for Audience Interest Prediction of News Topic on Social Media. In Proceedings of the International Conference on Extending Database Technology (EDBT2021), Nicosia, Cyprus, 23–26 March 2021; pp. 588–599. https://doi.org/10.5441/002/EDBT.2021.69.

[2] Mustafaraj, E.; Metaxas, P.T. The Fake News Spreading Plague. In Proceedings of the ACM on Web Science Conference, Troy, NY, USA, 25–28 June 2017; pp. 235–239. https://doi.org/10.1145/3091478.3091523.

[3] Ruths, D. The misinformation machine. Science **2019**, 363, 348–348. https://doi.org/10.1126/science.aaw1315.

[4] Bastos, M.T.; Mercea, D. The Brexit Botnet and User-Generated Hyperpartisan News. Soc. Sci. Comput. Rev. **2017**, 37, 38–54. https://doi.org/10.1177/0894439317734157.

[5] Bovet, A.; Makse, H.A. Influence of fake news in Twitter during the 2016 US presidential election. Nat. Commun. **2019**, 10, 7. https://doi.org/10.1038/s41467-018-07761-2.

[6] Rzymski, P.; Borkowski, L.; Drąg, M.; Flisiak, R.; Jemielity, J.; Krajewski, J.; Mastalerz-Migas, A.; Matyja, A.; Pyrć, K.; Simon, K.; et al. The Strategies to Support the COVID-19

Vaccination with Evidence-Based Communication and Tackling Misinformation. Vaccines **2021**, 9, 109. https://doi.org/10.3390/vaccines9020109.

[7] Truică, C.O.; Apostol, E.S.; Paschke, A. Awakened at CheckThat! 2022: Fake news detection using BiLSTM and sentence transformer. In Proceedings of the Working Notes of the Conference and Labs of the Evaluation Forum (CLEF2022), Bologna, Italy, 5–8 September 2022; pp. 749–757.

[8] European Commission. Fighting Disinformation; European Commission: Brussels, Belgium, 2020.

[9] Chen, Y.; Conroy, N.K.; Rubin, V.L. News in an online world: The need for an "automatic crap detector". Proc. Assoc. Inf. Sci. Technol. **2015**, 52, 1–4. https://doi.org/10.1002/pra2.2015.145052010081.

[10] Wang, W.Y. "Liar, Liar Pants on Fire": A New Benchmark Dataset for Fake News Detection. In Proceedings of the Annual Meeting of the Association for Computational Linguistics, Vancouver, BC, Canada, 30 July–4 August 2017; pp. 422–426. https://doi.org/10.18653/v1/P17-2067.

[11] Conroy, N.K.; Rubin, V.L.; Chen, Y. Automatic deception detection: Methods for finding fake news. Proc. Assoc. Inf. Sci. Technol. **2015**, 52, 1–4. https://doi.org/10.1002/pra2.2015.145052010082.

[12] Kaliyar, R.K.; Goswami, A.; Narang, P.; Sinha, S. FNDNet – A deep convolutional neural network for fake news detection. Cogn. Syst. Res. **2020**, 61, 32–44. https://doi.org/10.1016/j.cogsys.2019.12.005.

[13] Goldani, M.H.; Safabakhsh, R.; Momtazi, S. Convolutional neural network with margin loss for fake news detection. Inf. Process. Manag. **2021**, 58, 102418. https://doi.org/10.1016/j.ipm.2020.102418.

[14] Saleh, H.; Alharbi, A.; Alsamhi, S.H. OPCNN-FAKE: Optimized convolutional neural network for fake news detection. IEEE Access **2021**, 9, 129471–129489. https://doi.org/10.1109/ACCESS.2021.3112806.

[15] Samantaray, S.; Kumar, A. Bi-directional Long Short-Term Memory Network for Fake News Detection from Social Media. In Intelligent and Cloud Computing; Springer: Berlin/Heidelberg, Germany, 2022; pp. 463–470. https://doi.org/10.1007/978-981-16-9873-6_42.

[16] Ilie, V.I.; Truică, C.O.; Apostol, E.S.; Paschke, A. Context-Aware Misinformation Detection: A Benchmark of Deep Learning Architectures Using Word Embeddings. IEEE Access **2021**, 9, 162122–162146. https://doi.org/10.1109/ACCESS.2021.3132502.

[17] Jwa, H.; Oh, D.; Park, K.; Kang, J.; Lim, H. exBAKE: Automatic Fake News Detection Model Based on Bidirectional Encoder Representations from Transformers (BERT). Appl. Sci. **2019**, 9, 4062. https://doi.org/10.3390/app9194062.

[18] Kaliyar, R.K.; Goswami, A.; Narang, P. FakeBERT: Fake news detection in social media with a BERT-based deep learning approach. Multimed. Tools Appl. **2021**, 80, 11765–11788. https://doi.org/10.1007/s11042-020-10183-2.

[19] Kula, S.; Choraś, M.; Kozik, R. Application of the BERT-Based Architecture in Fake News Detection. In Conference on Complex, Intelligent, and Software Intensive Systems; Springer: Berlin/Heidelberg, Germany, 2020; pp. 239–249. https://doi.org/10.1007/978-3-030-57805-3_23.

[20] Mersinias, M.; Afantenos, S.; Chalkiadakis, G. CLFD: A Novel Vectorization Technique and Its Application in Fake News Detection. In Proceedings of the Language Resources and Evaluation Conference, Marseille, France, 11–16 May 2020; pp. 3475–3483.

[21] Mondal, S.K.; Sahoo, J.P.; Wang, J.; Mondal, K.; Rahman, M.M. Fake News Detection Exploiting TF-IDF Vectorization with Ensemble Learning Models. In Advances in Distributed Computing and Machine Learning; Springer: Berlin/Heidelberg, Germany, 2022; pp. 261–270. https://doi.org/10.1007/978-981-16-4807-6_25.

[22] Aslam, N.; Khan, I.U.; Alotaibi, F.S.; Aldaej, L.A.; Aldubaikil, A.K. Fake Detect: A Deep Learning Ensemble Model for Fake News Detection. Complexity **2021**, 2021, 5557784. https://doi.org/10.1155/2021/5557784.

[23] Truică, C.O.; Apostol, E.S. MisRoBÆRTa: Transformers versus Misinformation. Mathematics **2022**, 10, 569. https://doi.org/10.3390/ math10040569.

[24] Sedik, A.; Abohany, A.A.; Sallam, K.M.; Munasinghe, K.; Medhat, T. Deep fake news detection system based on concatenated and recurrent modalities. Expert Syst. Appl. **2022**, 208, 117953. https://doi.org/10.1016/j.eswa.2022.117953.

[25] Verma, P.K.; Agrawal, P.; Amorim, I.; Prodan, R. WELFake: Word Embedding Over Linguistic Features for Fake News Detection. IEEE Trans. Comput. Soc. Syst. **2021**, 8, 881–893. https://doi.org/10.1109/TCSS.2021.3068519.

[26] Shu, K.; Cui, L.; Wang, S.; Lee, D.; Liu, H. dEFEND: Explainable Fake News Detection. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, Anchorage, AK, USA, 4–8 August 2019; pp. 395–405. https://doi.org/10.1145/3292500.3330935.

[27] Khattar, D.; Goud, J.S.; Gupta, M.; Varma, V. MVAE: Multimodal Variational Autoencoder for Fake News Detection. In Proceedings of the World Wide Web Conference, San Francisco, CA, USA, 13–17 May 2019; pp. 2915–2921. https://doi.org/10.1145/3308558.3313552.

[28] Zhang, J.; Dong, B.; Yu, P.S. FakeDetector: Effective Fake News Detection with Deep Diffusive Neural Network. In Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE), Dallas, TX, USA, 20–24 April 2020; pp. 1826–1829. https://doi.org/10.1109/icde48307.2020.00180.

[29] Yang, S.; Shu, K.; Wang, S.; Gu, R.; Wu, F.; Liu, H. Unsupervised Fake News Detection on Social Media: A Generative Approach. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 27 January–1 February 2019; Voluume 33, pp. 5644–5651. https://doi.org/10.1609/aaai.v33i01.33015644.

[30] Wang, Y.; Qian, S.; Hu, J.; Fang, Q.; Xu, C. Fake News Detection via Knowledge-driven Multimodal Graph Convolutional Networks. In Proceedings of the 2020 International Conference on Multimedia Retrieval, Dublin, Ireland, 8–11 June 2020; pp. 540–547. https://doi.org/10.1145/3372278.3390713.

[31] Le, Q.; Mikolov, T. Distributed representations of sentences and documents. In Proceedings of the International Conference on Machine Learning PMLR, Bejing, China, 22–24 June 2014; pp. 1188–1196.

[32] Cui, J.; Kim, K.; Na, S.H.; Shin, S. Meta-Path-based Fake News Detection Leveraging Multi-level Social Context Information. In Proceedings of the Proceedings of the 31st ACM International Conference on Information & Knowledge Management, Atlanta, GA, USA, 17–21 October 2022; pp. 325–334.

[33] Singh, L. Fake news detection: A comparison between available Deep Learning techniques in vector space. In Proceedings of the 2020 IEEE 4th Conference on Information & Communication Technology (CICT), Chennai, India, 3–5 December 2020; pp. 1–4.

[34] Truică, C.O.; Apostol, E.S.; Darmont, J.; Assent, I. TextBenDS: A Generic Textual Data Benchmark for Distributed Systems. Inf. Syst. Front. **2021**, 23, 81–100. https://doi.org/10.1007/s10796-020-09999-y.

[35] Paltoglou, G.; Thelwall, M. A Study of Information Retrieval Weighting Schemes for Sentiment Analysis. In Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics, Stroudsburg, PA, USA, 11–16 July 2010; pp. 1386–1395.

[36] Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient estimation of word representations in vector space. In Proceedings of the Workshop Proceedings of the International Conference on Learning Representations 2013, Scottsdale, AZ, USA, 2–4 May 2013; pp. 1–12.

[37] Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G.S.; Dean, J. Distributed Representations of Words and Phrases and their Compositionality. In Proceedings of the Advances in Neural Information Processing Systems, Lake Tahoe, NV, USA, 5–10 December 2013; Volume 26, pp. 1–9.

[38] Bojanowski, P.; Grave, E.; Joulin, A.; Mikolov, T. Enriching word vectors with subword information. Trans. Assoc. Comput. Linguist. **2017**, 5, 135–146. https://doi.org/10.1162/tacl_a_00051.

[39] Pennington, J.; Socher, R.; Manning, C.D. GloVe: Global vectors for word representation. In Proceedings of the Conference on Empirical Methods in Natural Language Processing, Doha, Qatar, 25–29 October 2014; pp. 1532–1543. https://doi.org/10.3115/v1/D14-1162.

[40] Mikolov, T.; Grave, E.; Bojanowski, P.; Puhrsch, C.; Joulin, A. Advances in Pre-Training Distributed Word Representations. In Proceedings of the International Conference on Language Resources and Evaluation, Miyazaki, Japan, 7–12 May 2018; pp. 52–55.

[41] Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In Proceedings of the Conference of the North Association for Computational Linguistics, Minneapolis, MN, USA, 2–7 June 2019; pp. 4171–4186. https://doi.org/10.18653/v1/N19-1423.

[42] Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv **2019**, arXiv:1907.11692.

[43] Lewis, M.; Liu, Y.; Goyal, N.; Ghazvininejad, M.; Mohamed, A.; Levy, O.; Stoyanov, V.; Zettlemoyer, L. BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension. In Proceedings of the Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 7871–7880. https://doi.org/10.18653/v1/2020.acl-main.703.

[44] Wolf, T.; Debut, L.; Sanh, V.; Chaumond, J.; Delangue, C.; Moi, A.; Cistac, P.; Rault, T.; Louf, R.; Funtowicz, M.; et al. Transformers: State-of-the-Art Natural Language Processing. In Proceedings of the Conference on Empirical Methods in Natural Language Processing, Honolulu, HI, USA, 16–20 November 2020; pp. 38–45. https://doi.org/10.18653/v1/2020.emnlp-demos.6.

[45] Rennie, J.D.M.; Shih, L.; Teevan, J.; Karger, D.R. Tackling the Poor Assumptions of Naive Bayes Text Classifiers. In Proceedings of the International Conference on International Conference on Machine Learning, Los Angeles, CA, USA, 23–24 June 2003; pp. 616–623.

[46] Hochreiter, S.; Schmidhuber, J. Long Short-Term Memory. Neural Comput. **1997**, 9, 1735–1780. https://doi.org/10.1162/neco.1997.9. 8.1735.

[47] Cho, K.; van Merrienboer, B.; Bahdanau, D.; Bengio, Y. On the Properties of Neural Machine Translation: Encoder–Decoder Approaches. In Proceedings of the SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation, Doha, Qatar, 25 October 2014; pp. 103–111. https://doi.org/10.3115/v1/w14-4012.

[48] Hewamalage, H.; Bergmeir, C.; Bandara, K. Recurrent Neural Networks for Time Series Forecasting: Current status and future directions. Int. J. Forecast. **2021**, 37, 388–427. https://doi.org/10.1016/j.ijforecast.2020.06.008.

[49] Sokolova, M.; Lapalme, G. A systematic analysis of performance measures for classification tasks. Inf. Process. Manag. **2009**, 45, 427–437. https://doi.org/10.1016/j.ipm.2009.03.002.

[50] Szpakowski, M. FakeNewsCorpus. 2020. Available online: `https://github.com/several27/FakeNewsCorpus` (accessed on 27 December 2022).

[51] Upadhayay, B.; Behzadan, V. Sentimental LIAR: Extended Corpus and Deep Learning Models for Fake Claim Classification. In Proceedings of the 2020 IEEE International Conference on Intelligence and Security Informatics (ISI), virtual event, 9–10 November 2020; pp. 1–6. https://doi.org/10.1109/isi49825.2020.9280528.

[52] Horne, B.; Adali, S. This Just In: Fake News Packs A Lot In Title, Uses Simpler, Repetitive Content in Text Body, More Similar To Satire Than Real News. In Proceedings of the International AAAI Conference on Web and Social Media, Montreal, QC, Canada, 15–18 May 2017; pp. 759–766.

[53] Rashkin, H.; Choi, E.; Jang, J.Y.; Volkova, S.; Choi, Y. Truth of Varying Shades: Analyzing Language in Fake News and Political Fact-Checking. In Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing, Copenhagen, Denmark, 7–11 September 2017; pp. 2931–2937. https://doi.org/10.18653/v1/D17-1317.

[54] Barrón-Cedeño, A.; Jaradat, I.; Da San Martino, G.; Nakov, P. Proppy: Organizing the news based on their propagandistic content. Inf. Process. Manag. **2019**, 56, 1849–1864. https://doi.org/10.1016/j.ipm.2019.03.005.

[55] Kurasinski, L.; Mihailescu, R.C. Towards Machine Learning Explainability in Text Classification for Fake News Detection. In Proceedings of the 2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA), Miami, FL, USA, 14–17 December 2020; pp. 775–781. https://doi.org/10.1109/ICMLA51294.2020.00127.

[56] Nørregaard, J.; Horne, B.D.; Adalı, S. NELA-GT-2018: A Large Multi-Labelled News Dataset for the Study of Misinformation in News Articles. In Proceedings of the International AAAI Conference on Web and Social Media, Münich, Germany, 11–14 June 2019; pp. 630–638.

[57] Kwak, H.; An, J.; Ahn, Y.Y. A Systematic Media Frame Analysis of 1.5 Million New York Times Articles from 2000 to 2017. In Proceedings of the ACM Conference on Web Science, Southampton, UK, 6–10 July 2020; pp. 305–314. https://doi.org/10.1145/3394231.3397921.

[58] Reed, R.D.; Marks II, R.J. Neural Smithing: Supervised Learning in Feedforward Artificial Neural Networks; MIT Press: Cambridge, MA, USA, 1999.

[59] Arora, S.; Ge, R.; Moitra, A. Learning Topic Models – Going beyond SVD. In Proceedings of the Annual Symposium on Foundations of Computer Science, Washington, DC, USA, 20–23 October 2012; pp. 1–10. https://doi.org/10.1109/FOCS.2012.49.

[60] Bird, S.; Loper, E.; Klein, E. Natural Language Processing with Python; O'Reilly: Sebastopol, CA, USA, 2009.

[61] Pedregosa, F.; Varoquaux, G.; Gramfort, A.; Michel, V.; Thirion, B.; Grisel, O.; Blondel, M.; Prettenhofer, P.; Weiss, R.; Dubourg, V.; et al. Scikit-learn: Machine Learning in Python. J. Mach. Learn. Res. **2011**, 12, 2825–2830.

[62] Grootendorst, M. PolyFuzz. 2020. Available online: `https://maartengr.github.io/PolyFuzz/` (accessed on 27 December 2022).

[63] Řehůřek, R.; Sojka, P. Software Framework for Topic Modelling with Large Corpora. In Proceedings of the Workshop on New Challenges for NLP Frameworks, Valletta, Malta, 22 May 2010, pp. 45–50.

[64] Kula, M. Python-Glove. 2020. Available online: `https://github.com/maciejkula/glove-python` (accessed on 27 December 2022).

[65] Rajapakse, T. SimpleTransformers. 2021. Available online: https://simpletransformers.ai/ (accessed on 27 December 2022).

[66] Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, Hong Kong, China, 3–7 November 2019; pp. 3982–3992. https://doi.org/10.18653/v1/D19-1410.

[67] Chen, T.; Guestrin, C. XGBoost: A Scalable Tree Boosting System. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, 13–17 August 2016; pp. 785–794. https://doi.org/10.1145/2939672.2939785.

[68] D'Ulizia, A.; Caschera, M.C.; Ferri, F.; Grifoni, P. Fake news detection: A survey of evaluation datasets. PeerJ Comput. Sci. **2021**, 7, e518. https://doi.org/10.7717/peerj-cs.518.

[69] Alhindi, T.; Petridis, S.; Muresan, S. Where is Your Evidence: Improving Fact-checking by Justification Modeling. In Proceedings of the First Workshop on Fact Extraction and VERification (FEVER), Brussels, Belgium, 24 July 2018; pp. 85–90. https://doi.org/10.18653/v1/W18-5513.

[70] Barrón-Cedeño, A.; Martino, G.D.S.; Jaradat, I.; Nakov, P. Proppy: A System to Unmask Propaganda in Online News. In Proceedings of the AAAI Conference on Artificial Intelligence, Honolulu, HI, USA, 27 January–1 February 2019; pp. 9847–9848. https://doi.org/10.1609/aaai.v33i01.33019847.

[71] Truică, C.O.; Apostol, E.S.; Șerban, M.L.; Paschke, A. Topic-Based Document-Level Sentiment Analysis Using Contextual Cues. Mathematics **2021**, 9, 2722. https://doi.org/10.3390/math9212722.